MAHARASHTRA STATE BOARD OF TECHNICAL EDUCATION, MUMBAI

A Project Report on

Online Multiuser Vehicle Rental and Listing Platform

Submitted as a partial fulfilment for the award of

Diploma in Computer Engineering

Submitted By

Piyush Prashant Bhamare

Hatim Huzefa Bohri

Kartik Bhagwan Chaudhari

Chetan Jagdish Patel

DEPARTMENT OF COMPUTER ENGINEERING

S.S.V.P.S's Bapusaheb Shivajirao Deore Polytechnic, Dhule (Institute Code : 00590

2025-2026

A Project Report on

Online Multiuser Vehicle Rental and Listing Platform

Submitted as a partial fulfillment for the award of

Diploma in Computer Engineering

Submitted by

Piyush Prashant Bhamare	23610960176
Hatim Huzefa Bohri	23610960179
Kartik Bhagwan Chaudhari	23610960183
Chetan Jagdish Patel	23610960192

Guided by.

Ms. S. S. Hajari

DEPARTMENT OF COMPUTER ENGINEERING

S.S.V.P.S's Bapusaheb Shivajirao Deore Polytechnic, Dhule

(Institute Code : 0059)

2025 – 2026

MAHARASHTRA STATE BOARD OF TECHNICAL EDUCATION, MUMBAI

S.S.V.P.S's Bapusaheb Shivajirao Deore Polytechnic, Dhule

DEPARTMET OF COMPUTER ENGINEERING

CERTIFICATE

This is to certify that, Mr. / Ms. …… bearing examination seat number ……... has satisfactorily completed project entitled "……..……..……." along with batchmates in partial fulfillment for the **Diploma Course in Computer Engineering** of the **Maharashtra State Board of Technical Education** at our polytechnic during the academic year **2025-2026**. The project is completed by a group consisting of four persons under the guidance of the Faculty Guide.

Date : **Examination Seat No.:**

Place:

Signature: **Signature:**

Name : **Name :**

Faculty Guide **External Examiner**

Signature: **Signature:**

Name : **Name :**

H.O.D. **Principal**

MAHARASHTRA STATE BOARD OF TECHNICAL EDUCATION, MUMBAI

S.S.V.P.S's Bapusaheb Shivajirao Deore Polytechnic, Dhule

DEPARTMET OF COMPUTER ENGINEERING

CERTIFICATE

This is to certify that,

Sr. No.	Roll No.	Name	Enrollment Number	Exam Seat number
1	06	Piyush Prashant Bhamare	23610960176	405569
2	09	Hatim Huzefa Bohri	23610960179	405572
3	12	Kartik Bhagwan Chaudhari	23610960183	405575
4	20	Chetan Jagdish Patel	23610960192	405583

has satisfactorily completed project entitled "GoWheels- Online Multiuser Vehicle Rental and Listing Platform" in partial fulfillment for the **Diploma Course in Computer Engineering** of the **Maharashtra State Board of Technical Education** at our polytechnic during the academic year **2025-2026**. The project is completed by a group consisting of four persons under the guidance of the Faculty Guide.

Date :

Place: Dhule

Signature:

Name : Ms. S. S. Hajari

Faculty Guide

Signature:

Name :

External Examiner

Signature:

Name : Mr. N. D. Patel

Signature:

Name : Prof. S. A. Hire

H.O.D. **Principal**

Chapter 1
Introduction

Transportation plays a vital role in today's socio-economic environment. As cities continue to grow and travel needs become more dynamic, many people require access to vehicles without the long-term cost and responsibility of ownership. Traditionally, individuals looking for a car or bike whether for a short trip, an urgent business need, or as a temporary replacement have had to deal with a rental market that is often fragmented and poorly organized. This old traditional system typically lacks transparency, with inconsistent pricing, limited options, and a heavy reliance on manual paperwork. In addition, independent vehicle owners and small rental agencies face challenges in managing bookings, verifying customer identities, and ensuring the safety and proper use of their vehicles.

This is where "**GoWheels**" an Online Multiuser Vehicle Rental and Listing Platform bridges the gap. GoWheels is a web-based application developed using Python and the Django framework. It works as a **two-sided marketplace** that connects vehicle owners, who want to earn from their unused vehicles as well as rental agencies, with users who need convenient and reliable transportation. The main goal of GoWheels is to make the vehicle rental process simple, efficient, and fully digital, replacing the traditional offline methods.

GoWheels is designed to reduce common problems in the rental market by introducing multiple useful features. For example, it includes secure **digital verification** of important documents such as Aadhaar cards and driving licenses to ensure trust and safety between users. GoWheels also provides flexibility by allowing customers to either rent vehicles for self-driving or hire a verified driver through the platform. To make payments easier, it offers a **built-in digital wallet** along with multiple payment options like **UPI** and **Cash on Pickup**. By solving issues related to trust, availability, and manual processes, GoWheels creates a transparent system where users can clearly see vehicle options, pricing, and rental terms from booking to return.

The vehicle rental industry in India is currently experiencing rapid growth and transformation. In the past, the sector was mainly controlled either by large companies with significant financial investments or by small, unorganized local operators working through offline methods. Through GoWheels, this project aims to contribute to this transformation by providing a

structured, transparent, and user-friendly digital platform that simplifies vehicle rentals for both owners and users.

Recent industry data shows strong growth in this sector, driven by changes in the economy, increasing disposable income, and shifting consumer behaviour. Today's consumers, especially millennials and Gen Z, prefer using services when needed rather than owning assets permanently.

Table 1: India Car Rental Market Overview

Market Metric	Data / Valuation
Estimated Market Size (2025)	₹2,60,586.8 Million (≈ ₹26,058.7 Crore)
Projected Market Size (2034)	₹8,32,481.7 Million (≈ ₹83,248.2 Crore)
Compound Annual Growth Rate (CAGR)	13.78% (2026–2034)
Top Revenue Segments	Economy Cars, Executive, & SUVs
Dominant Booking Mode	Online / App-based Bookings

The growth of internet access, along with the increasing use of digital wallets and UPI payments, has significantly changed the way vehicle rentals are booked. Most users now prefer quick and convenient online booking methods. This digital shift is not limited to major cities; it is also rapidly spreading to Tier-2 and Tier-3 cities, making vehicle rental services more accessible to a wider range of people.

Table 2: Key Factors Driving the Shift from Ownership to Rental

Factor	Impact on Consumer Behaviour
Financial Flexibility	Eliminates high down payments, EMIs, and rapid depreciation costs associated with owning a personal vehicle.
Variety and Choice	Renters can choose vehicles based on specific needs (e.g., SUV for trips, hatchback for city driving).
Urban Constraints	Solves parking issues and space limitations in congested urban areas.
Technological Convenience	App-based platforms enable instant booking, digital KYC, and seamless payments without physical visits.

Developing a platform like GoWheels requires addressing these modern demands using robust back-end frameworks like Python and Django to manage complex databases involving users, vehicles, drivers, rentals, reviews, and secure transactions.

Chapter 2
Literature Survey

2.1 Study of Existing System

To understand the practical challenges of the vehicle rental market, we conducted a study of local vehicle rental agencies and independent vehicle owners operating in and around Dhule city. Currently, the local market relies heavily on offline networks, manual management, and fragmented digital communication (like WhatsApp messages or calls).

Table 3: Existing Vehicle Rental Systems in India

Platform Name	Service Type	Key Features	Limitations
Zoomcar	Self-drive car rental	App-based booking, digital KYC, hourly/daily rentals	Limited availability in smaller cities
Revv	Self-drive & subscription	Doorstep delivery, flexible plans	Focused mainly on metro cities
Ola Drive	Self-drive rental	Integrated with Ola app, easy booking	Limited-service coverage
MyChoize	Self-drive & chauffeur	Corporate rentals, multiple vehicle options	Less penetration in tier-2 cities
Drivezy	Peer-to-peer rental	Vehicle sharing model, bikes & cars	Reduced operations in recent years
Savaari	Chauffeur-driven	Intercity travel, professional drivers	No self-drive option

Rapido Rentals	Bike rental	Affordable, quick booking	Limited to bikes and select cities

While discussing with these local agency owners and independent operators, the following questions were asked:

Q1. How do you maintain records of your vehicles and track their current availability?

Q2. How do customers discover your available vehicles and make bookings?

Q3. How do you verify customer identities and driving licenses before handing over the keys?

Q4. How are payments, security deposits, and rental durations calculated and recorded?

Q5. How do you allocate drivers if a customer requests a chauffeur-driven vehicle?

Q6. How do you schedule and track routine vehicle maintenance to ensure fleet safety and reliability?

Q7. What is your protocol for inspecting vehicles upon return, and how do you handle damage disputes with customers?

Q8. How does your business manage late returns or last-minute trip extensions without disrupting subsequent bookings?

Q9. What support systems or protocols are in place for customers experiencing on-road emergencies or vehicle breakdowns during their rental?

Q10. How do you process booking cancellations, and what is your policy regarding refunds or retaining security deposits?

Q11. How are insurance policies managed for your fleet, and what is the liability process in the event of an accident?

Q12. What specific software tools, spreadsheets, or manual ledgers are you currently using to manage your day-to-day operations?

Q13. How do you currently collect customer feedback, and what strategies do you use to encourage repeat bookings and build loyalty?

After discussing with them, we analyzed the following workflow of the existing system:

i. **Manual Record-Keeping:** Most owners use physical diaries, notebooks, or basic Excel spreadsheets to track which vehicle is currently rented, to whom, and its expected return date.

ii. **Fragmented Bookings:** There is no centralized catalog. Customers discover available cars or bikes primarily through word-of-mouth or physical visits.

iii. **Physical Document Storage:** To verify a user, owners collect physical photocopies of the customer's Aadhaar card and Driving License.

iv. **Unorganized Payments:** Payments and security deposits are handled in cash or direct UPI transfers, but the accounting is done manually without a dedicated ledger.

v. **Disconnected Driver Allocation:** If a customer needs a driver, the agency owner has to manually call their contacts to check driver availability and negotiate rates.

Table 4: Survey of Operational Methods Used by Local Rental Providers

Operational Task	Primary Method Used by Owners	Percentage of Owners
Inventory & Availability Tracking	Physical Registers / Diaries	68%
	Basic Excel Spreadsheets	24%
	Memorization / No Formal Tracking	8%
Customer Discovery & Booking	Word of Mouth / Walk-ins	45%
	WhatsApp Groups / Status	40%
	Justdial / Local Directories	15%
Document Verification (KYC)	Physical Photocopies (Stored in files)	85%
	Digital Photos on Phone Gallery	15%
Payment & Deposit Management	Cash Transactions	60%
	Direct UPI (Untracked ledger)	40%
Driver Allocation	Manual Phone Calls to Driver Network	100%

2.2 Limitations of Existing System

The manual record-keeping and fragmented communication for vehicle rentals come with several limitations that significantly impact both operational efficiency and customer satisfaction:

i. **Inefficient Record Keeping:** Vehicle availability, customer details, and rental durations are clubbed together in physical registers. This makes it difficult to quickly check if a specific vehicle is available for a future date.

ii. **Data Security Risks:** Storing physical photocopies of sensitive user documents (Aadhaar, Driving License) is insecure and risks loss, damage, or misuse of customer identity data.

iii. **Time-Consuming Verification:** Manually checking documents, calculating daily rental charges, and tracking down pending security deposits is highly inefficient and prone to human error.

iv. **Lack of Trust and Analysis:** The existing system lacks a historical database of customer behaviour or vehicle condition. Host cannot track bad renters, and renters cannot view reviews or ratings for vehicles or drivers before booking.

v. **Payment Disputes:** Without a centralized digital wallet or automated receipt system, disputes over advance payments, overtime fines, and fuel charges frequently occur.

Table 5: Time-Motion Analysis of a Standard Manual Booking

Task in Existing System	Average Time Taken (Manual)	Bottleneck / Limitation
Finding Available Vehicle	45 – 60 minutes	Requires calling multiple owners; no central inventory
Identity Verification	15 – 20 minutes	Physical document checking; risk of fake IDs
Driver Sourcing (If needed)	30 – 90 minutes	Manual calls to drivers; availability uncertainty and negotiation delays
Payment & Ledger Entry	10 minutes	Manual calculation of rental duration, advance, and deposit
Total Onboarding Time	**~ 1.5 to 3 Hours**	**Highly inefficient for urgent or short-term rentals**

2.3 Problem Identification

Problems for Vehicle Owners:

i. Host are unable to reach a wider digital audience and are restricted to their local physical network.

ii. Host face difficulty in tracking the availability of multiple vehicles across different dates.

iii. In spite of keeping physical records, host cannot take informed decisions based on historical data (e.g., identifying which vehicle generates the most profit).

iv. Risk of handing over expensive vehicles to unverified customers due to lack of standard KYC (Know Your Customer) processes.

Problems for Renters (Users):

i. Renters cannot seamlessly filter or search for specific vehicle categories (e.g., SUVs, Hatchbacks, Cruiser bikes) based on their immediate needs.

ii. Renters face a lack of transparency in pricing and often encounter hidden charges.

iii. Renters are unable to easily find a trusted, verified driver if they do not wish to self-drive.

iv. Tracking the total rental expense, security deposits, and booking history is entirely manual for the user.

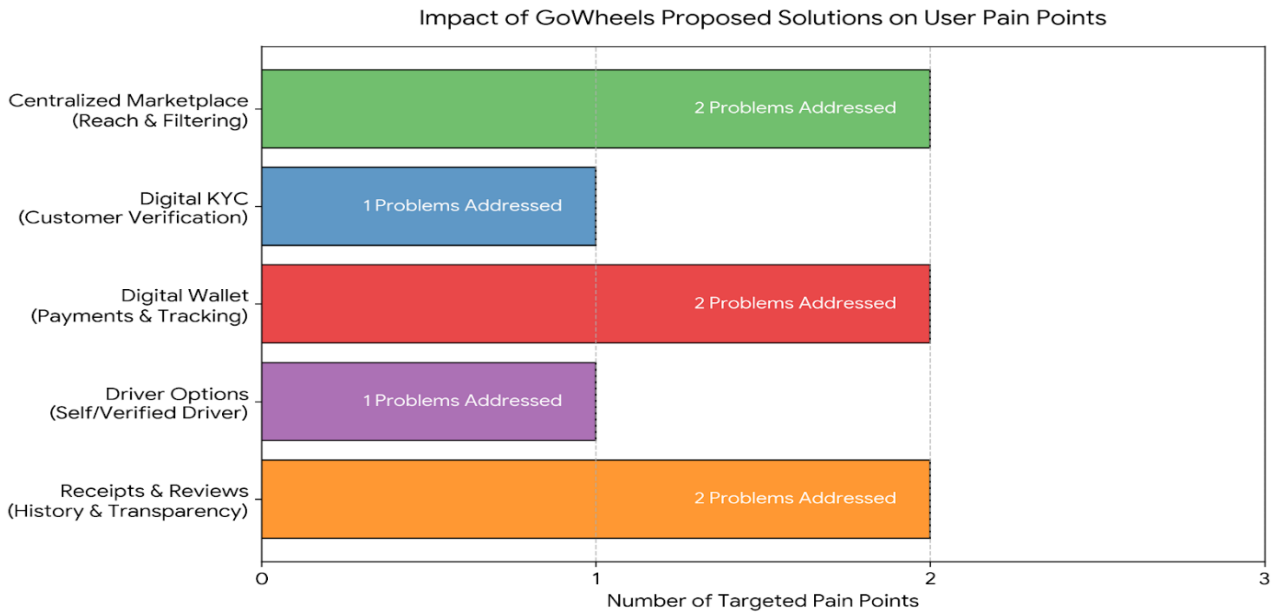

Fig 2.1: Graph of Impact of GoWheels Proposed Solutions on User Pain Points

2.4 Problem Definition

To design and develop a web-based application, **"GoWheels,"** that will facilitate seamless digital record-keeping and booking management for the vehicle rental industry.

To achieve this, the proposed system will provide:

i. A centralized digital marketplace for listing and discovering vehicles based on specific categories, fuel types, and seating capacities.
ii. Provision to digitally upload and verify user documents (Aadhaar and Driving License).
iii. An integrated digital Wallet system to securely manage transactions, credits, and debits.
iv. Options for users to choose between "Self-Drive" or hiring a "Verified Driver" within the same booking lifecycle.
v. Generation of transparent rental receipts and a comprehensive review system for vehicles and drivers.

This system will guide both vehicle host and renters in conducting secure, efficient, and transparent transactions, thereby upgrading the traditional rental process into a modernized digital ecosystem.

Chapter 3
Scope of the Project

The scope of the GoWheels covers the design, development, and deployment of a fully functional, web-based vehicle rental marketplace. The platform is designed using the Python Django framework to serve as a centralized digital bridge between vehicle host, individuals seeking short-term transportation, and independent drivers. The system is bounded by specific functional and operational parameters intended to solve the immediate limitations of the localized, manual rental industry while remaining scalable for future expansion.

1. Target Audience and System Roles

The platform is designed to cater to three primary categories of users:

i. **Renters (Customers):** Individuals looking for a convenient, transparent way to search, compare, and book vehicles (two-wheelers or four-wheelers) for temporary use.

ii. **Vehicle Owners (Hosts/Agencies):** Independent vehicle owners or local rental agencies who wish to list their idle vehicles on a centralized platform to reach a broader customer base and manage their inventory digitally.

iii. **Administrators:** The platform managers who oversee the entire ecosystem, verify user documents, resolve disputes, and monitor financial transactions to ensure a secure environment.

2. Functional Scope

The core functionality of the GoWheels system is divided into several integrated modules:

i. **Centralized Vehicle Cataloging:** The system allows host to list vehicles with detailed specifications, including vehicle category (e.g., SUV, Hatchback, Cruiser), fuel type, seating capacity, pricing per day, and high-quality images.

ii. **Dynamic Booking Engine:** Renters can search for vehicles based on availability dates. The system dynamically updates the inventory, preventing double-booking errors and displaying real-time availability.

iii. **Dual Rental Modes:** The platform explicitly supports two modes of renting: "Self-Drive" for users who prefer to drive themselves, and "With Driver" for users who require a chauffeur.

iv. **Digital KYC and Verification:** To ensure the security of the expensive assets being rented, the system includes a secure upload portal for mandatory identity verification, specifically capturing the user's Aadhaar Card and Driving License.

v. **Integrated Wallet and Financial Tracking:** A built-in digital wallet system tracks all financial transactions. It manages advanced payments, total trip calculations, security deposits, and final settlements, providing a transparent ledger for both the owner and the renter.

vi. **Feedback and Rating Mechanism:** Post-rental, the system facilitates a dual-review process. Renters can rate the vehicle's condition and the driver's behavior, while host can rate the renter, fostering a community built on trust and accountability.

3. Operational and Technical Scope

i. **Platform Accessibility:** GoWheels is developed as a responsive web application. It is designed to be accessible across various devices (desktops, laptops, and mobile browsers) without requiring users to download a dedicated mobile application.

ii. **Data Security:** The system prioritizes the secure handling of sensitive user data, ensuring that uploaded KYC documents are stored securely and accessed only by authorized administrators for verification purposes.

iii. **Geographical Boundary:** While the digital architecture is highly scalable, the initial operational scope of the project is targeted at localized markets (such as Dhule city and surrounding regions) to test and refine the peer-to-peer rental model before scaling to a state or national level.

4. Out of Scope (Current Limitations)

To maintain project feasibility within the academic timeframe, certain advanced features are currently considered out of scope but remain open for future development. The current system does not include:

i. In-app automated insurance claim processing in the event of an accident.

ii. Integration with live traffic or route mapping APIs for trip tracking.

iii. Integration with live third-party payment gateways (e.g., Razorpay, Stripe) for direct bank transfers, relying instead on the internal virtual wallet simulation.

Chapter 4
Methodology

4.1 Requirements Gathering and Analysis

To ensure the successful development of the GoWheels platform, a structured Agile/Iterative development methodology was adopted. Given the dynamic nature of web applications and the need to accommodate both vehicle host and renters, this approach allowed the team to gather initial requirements, build core modules, and continuously refine the system based on testing and feedback. The most crucial phase of this methodology was Requirements Gathering and Analysis which involved defining exactly what the GoWheels system needs to do.

A. Requirement Gathering Techniques

To understand the exact needs of the end-users, the following techniques were utilized:

- **Field Interviews:** Informal interviews were conducted with local vehicle rental agency owners and independent drivers to understand manual workflows, pricing models, and pain points regarding security deposits and driver allocation.
- **Analysis of Existing Systems:** A comprehensive study of existing car-pooling and corporate rental platforms was conducted to identify standard UI/UX patterns.
- **Brainstorming Sessions:** Team discussions were held to finalize the technical stack and outline the database schema required to handle dual-user roles effectively.

B. System Requirements (Hardware and Software)

Based on the analysis, the following technical requirements were established for the development and deployment of the GoWheels application:

Table 6: Software Requirements

Component	Technology / Tool Used
Operating System	Windows 10/11
Programming Language	Python (Version 3.8 or higher)
Backend Framework	Django (Version 4.x or higher)
Frontend Technologies	HTML5, CSS3, JavaScript, Bootstrap 5

Database	SQLite
IDE / Code Editor	Visual Studio Code (VS Code) or PyCharm
Version Control	Git and GitHub

4.2 User Requirements (Functional Requirements)

There are two primary user roles for the GoWheels system: Host (Vehicle Owner) and User (Renter/Customer).

A. Requirements of Host/Owner:

i. Host shall be able to securely log in with their unique username and password to access the fleet dashboard.
ii. Host shall be able to add, update, and remove vehicles from the platform's inventory, including uploading specifications and vehicle images.
iii. Host shall be allowed to review, approve, or reject user KYC document submissions (Aadhaar Card and Driving License).
iv. Host shall be able to monitor and manage the complete lifecycle of all bookings (Pending, Approved, Completed, Cancelled) and explicitly apply/manage late return penalties.
v. Host shall be able to view financial analytics, track revenue, and oversee user security deposits.
vi. Host shall be able to read and monitor ratings and descriptive feedback left by users for specific vehicles and drivers.

B. Requirements of User/Renter:

i. The user shall be able to register, log in, and reset their password securely via an account management system.
ii. The user shall be allowed to upload images of their Aadhaar Card and Driving License for mandatory KYC verification before booking.
iii. The user shall be able to view, search, and filter available vehicles based on category, fuel type, seating capacity, etc.
iv. The user shall be able to utilize a booking engine to select pick-up and drop-off dates and check real-time vehicle availability.
v. The user shall be allowed to choose between "Self-Drive" or "With Driver" options during the checkout process.

vi. The user shall be able to manage a digital wallet to view their balance, track transaction history, pay for trips, and manage security deposits.

vii. The user shall be able to submit a rating and descriptive review evaluating the vehicle condition and driver behaviour after a trip is completed.

viii. The User shall be able to utilize a Helpdesk/Support portal to log issues or request assistance regarding their bookings.

4.3 Non-Functional Requirements

i. **Security & Authentication:** Implement robust user authentication and authorization mechanisms, utilizing cryptographic hashing for passwords to protect sensitive project and user data.

ii. **Vulnerability Protection:** The system must be protected against standard web vulnerabilities, including Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), and SQL Injection attacks.

iii. **Data Integrity:** The system should enforce strict data integrity rules at the database level to dynamically prevent a single vehicle from being double-booked for overlapping dates.

iv. **Performance:** The system should deliver high performance by optimizing and caching vehicle images and processing search queries efficiently without noticeable lag.

v. **Usability & UI:** The system should feature a fully responsive frontend design, ensuring seamless usability across both mobile devices and desktop computers. It must be highly intuitive, allowing users with minimal technical experience to navigate and complete bookings easily.

vi. **Scalability:** The system architecture must be scalable to accommodate an increasing catalog of vehicles, growing user registration, and higher concurrent traffic without performance degradation.

vii. **Maintainability:** The codebase should follow a structured architectural pattern (e.g., Model-Template-View) to ensure that future updates, bug fixes, or feature expansions can be implemented efficiently.

Chapter 5
Details of designs, working and processes

5. System Design:

For GoWheels, the System Design phase involved creating a robust architecture, database structure, and user interface that catered specifically to the dual-role requirements gathered from vehicle host and renters.

Here, we translated requirements into a comprehensive system design that outlines the application's overall structure, logical components, and functionality.

Firstly, we defined the relational database schema to efficiently store user data, vehicle inventory, and transaction records. We identified the necessary data entities such as User Profiles (e.g., KYC documents, wallet balance, contact details), Vehicle Inventory (e.g., vehicle category, fuel type, seating capacity, pricing, availability status), Booking/Rental records (e.g., start and end dates, payment modes, late penalty tracking), and Driver records (e.g., allocations, ratings, daily fees).

Based on the identified requirements, we defined the core functionalities of the application such as:

i. User Authentication & Profile Management
ii. Vehicle Inventory Management
iii. Driver Allocation & Management
iv. Booking & Reservation Engine
v. Wallet & Payment Management
vi. Feedback & Review Management
vii. Fleet Analytics & Reporting Dashboard

A. Algorithm

Input:

User actions (registration, login, booking, listing vehicles, payments, etc.)

Output:

Successful vehicle booking, vehicle listing, transaction completion, and user interactions stored in the system

Procedure:

i.System Initialization

i. Start the system

ii. Load the homepage interface

iii. Wait for user interaction

II.User Authentication Process

i. When a user visits the homepage, check login status

ii. If the user is already authenticated, redirect to the dashboard

iii. If the user is not authenticated, display authentication options

iv. If the user selects **Login**:

a. Prompt user to enter credentials (email/username and password)

b. Validate credentials with database records

c. If valid, grant access and redirect to dashboard

d. If invalid, display error and allow retry

v. If the user selects **Sign Up**:

a. Collect user details (username, email, password, etc.)

b. Validate input data

c. Create new user account in database

d. Redirect user to login page

vi. If the user selects **Forgot Password**:

a. Prompt user to request password reset

b. Send reset instructions (OTP/email)

c. Allow user to set a new password

d. Redirect user to login page

vii. After successful authentication, load the user dashboard

III.Customer Flow: Vehicle Booking

i. From dashboard, navigate to Vehicles or Offers page

ii. Retrieve and display available vehicles from database

iii. Allow user to filter/search vehicles based on preferences

iv. User selects a vehicle

v. Redirect to rental form

vi. Collect rental details:

a. Start date and end date

b. User age

c. Identity details (Aadhaar, license)

vii. Check if user requires a driver:

a. If **Yes**, display available drivers with ratings

b. Allow user to select a driver

c. If **No**, proceed with self-drive option

viii. Provide option to apply promo code:

a. If entered, validate promo code

b. Apply discount if valid

ix. Calculate total rental cost based on duration, vehicle, and driver (if selected)

x. Redirect to payment page

xi. Allow user to select payment method:

a. Wallet

b. Online payment gateway

c. Cash (if applicable)

xii. Process payment:

a. If successful, confirm booking

b. If failed, notify user and allow retry

xiii. Store booking details in database

xiv. Update vehicle availability

xv. Redirect user to rental history page

xvi. Allow user to submit review and rating (optional)

IV.Owner Flow: Vehicle Listing and Management

i. From dashboard, navigate to List Vehicle page

ii. Display vehicle listing form

iii. Collect vehicle details:

a. Category and type

b. Fuel type

c. Number of seats

d. Vehicle number and pricing

iv. Upload vehicle images

v. Validate all inputs

vi. Save vehicle data in database

vii. Confirm successful vehicle listing

viii. Redirect to “Your Vehicles” page

ix. Allow owner to select a listed vehicle

x. Provide options to:

a. Edit vehicle details

b. Update availability status

xi. Save updated information in database

V.Support and Help Services

i. Allow user to navigate to Helpdesk page

ii. Provide options to:

a. View FAQs

b. Submit support ticket

iii. Store support requests in system

iv. Notify user of submission status

VI.System Termination

i. Continue system operation for multiple users

ii. End process when user exits the application

B. Flowchart

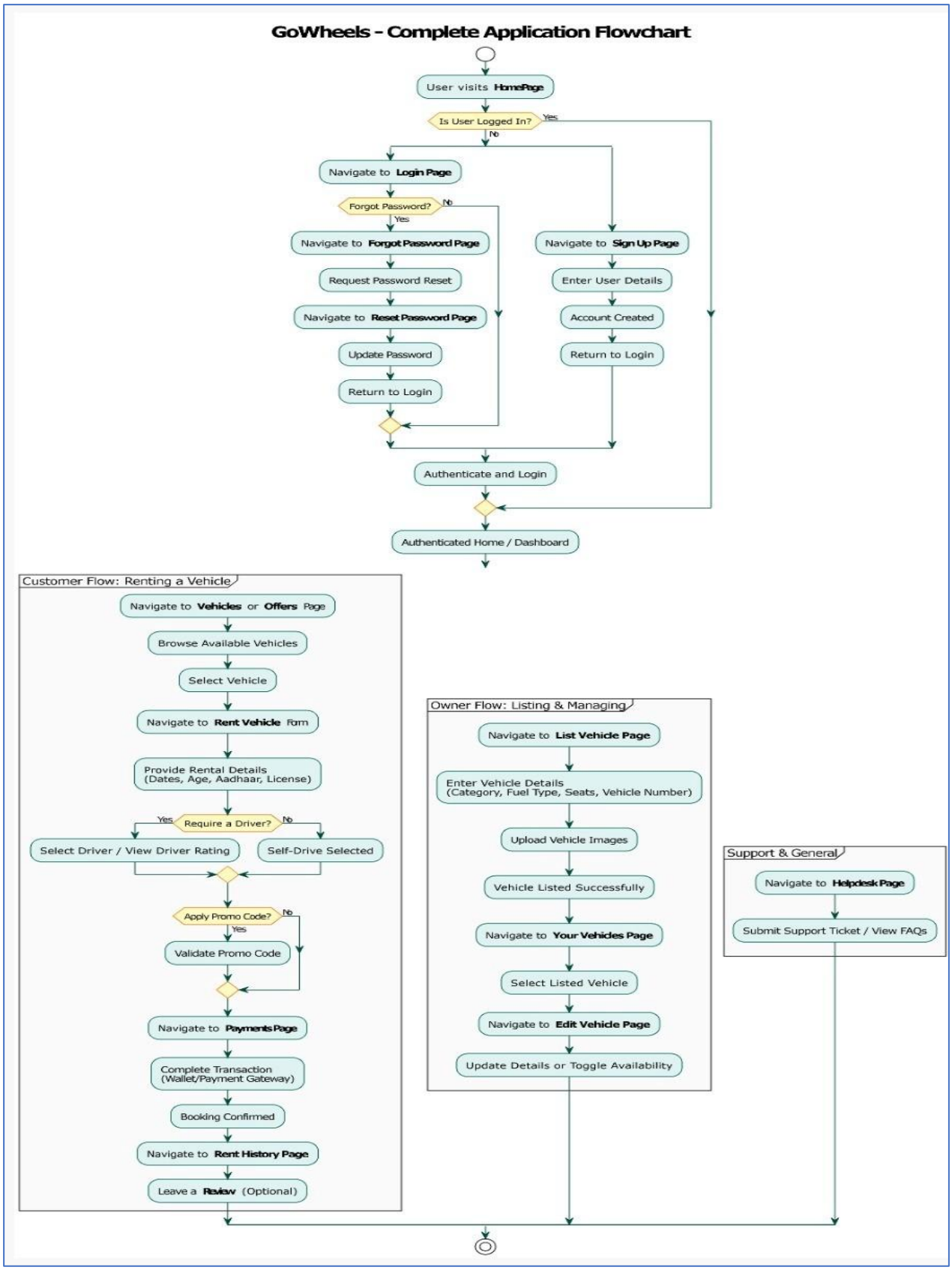

Fig.5.1 Flowchart of System

C. System Diagram

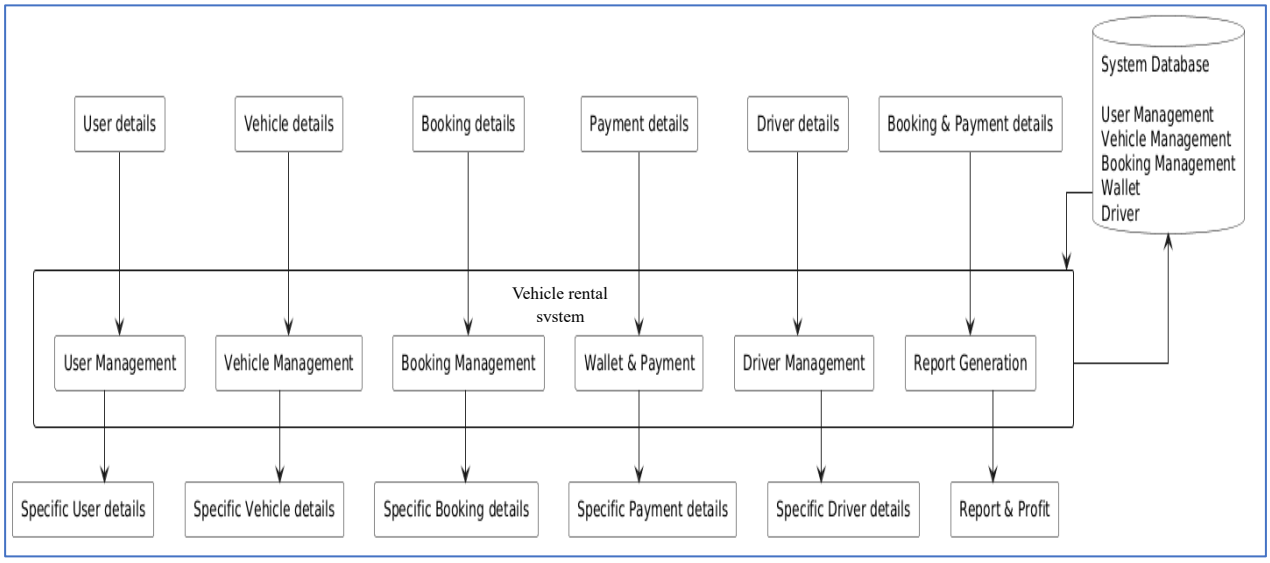

Fig.5.2 System Diagram

The system architecture of GoWheels represents the overall structure and interaction between different components involved in the vehicle rental platform. It follows a **three-tier architecture** consisting of the Frontend (Presentation Layer), Backend (Application Layer), and Database (Data Layer), along with integration of external services.

1. Presentation Layer (Frontend)

The presentation layer is responsible for user interaction and interface rendering. It is developed using Django Templates (HTML, CSS, JavaScript).

Users interact with the system through web pages to perform actions such as:

i. User registration and login with OTP verification
ii. Searching and filtering vehicles
iii. Booking vehicles and selecting drivers
iv. Making payments (wallet, online, or cash)
v. Viewing rental history and submitting reviews
vi. This layer sends user requests to the backend and displays the processed results.

2. Application Layer (Backend)

The backend is built using the Django Framework, which handles all business logic and application processing. It consists of multiple functional modules:

i. **Authentication Module:** Manages user signup, login, OTP verification, and password reset.

ii. **Vehicle Module:** Handles vehicle listing, searching, filtering, and management by owners.

iii. **Rental Module:** Processes booking requests, checks availability, calculates pricing, and manages rental records.
iv. **Payment Module:** Integrates with external payment gateway (Stripe) to handle online transactions.
v. **Wallet Module:** Maintains user wallet balance, handles recharge and payment deductions.
vi. **Review Module:** Allows users to submit ratings and feedback for vehicles and drivers.
vii. **Email Service Module:** Sends booking confirmation emails and notifications to users.

3. Data Layer (Database)

The system uses a relational database (SQLite/MySQL) to store and manage data. The main entities include:

i. **User:** Stores user credentials and profile details.
ii. **Vehicle:** Contains vehicle information such as type, price, and availability.
iii. **Driver:** Stores driver details and ratings.
iv. **Rental:** Maintains booking records, dates, and payment information.
v. **Wallet & Transactions:** Tracks user balance and transaction history.
vi. **Review:** Stores feedback and ratings given by users.

4. External Services Integration

The system integrates with external services to enhance functionality:

- **Stripe Payment Gateway:** Used for secure online payments during vehicle booking and wallet recharge.
- **Email Service:** Sends automated booking confirmation emails and OTP verification messages to users.

D. Use Case Diagram

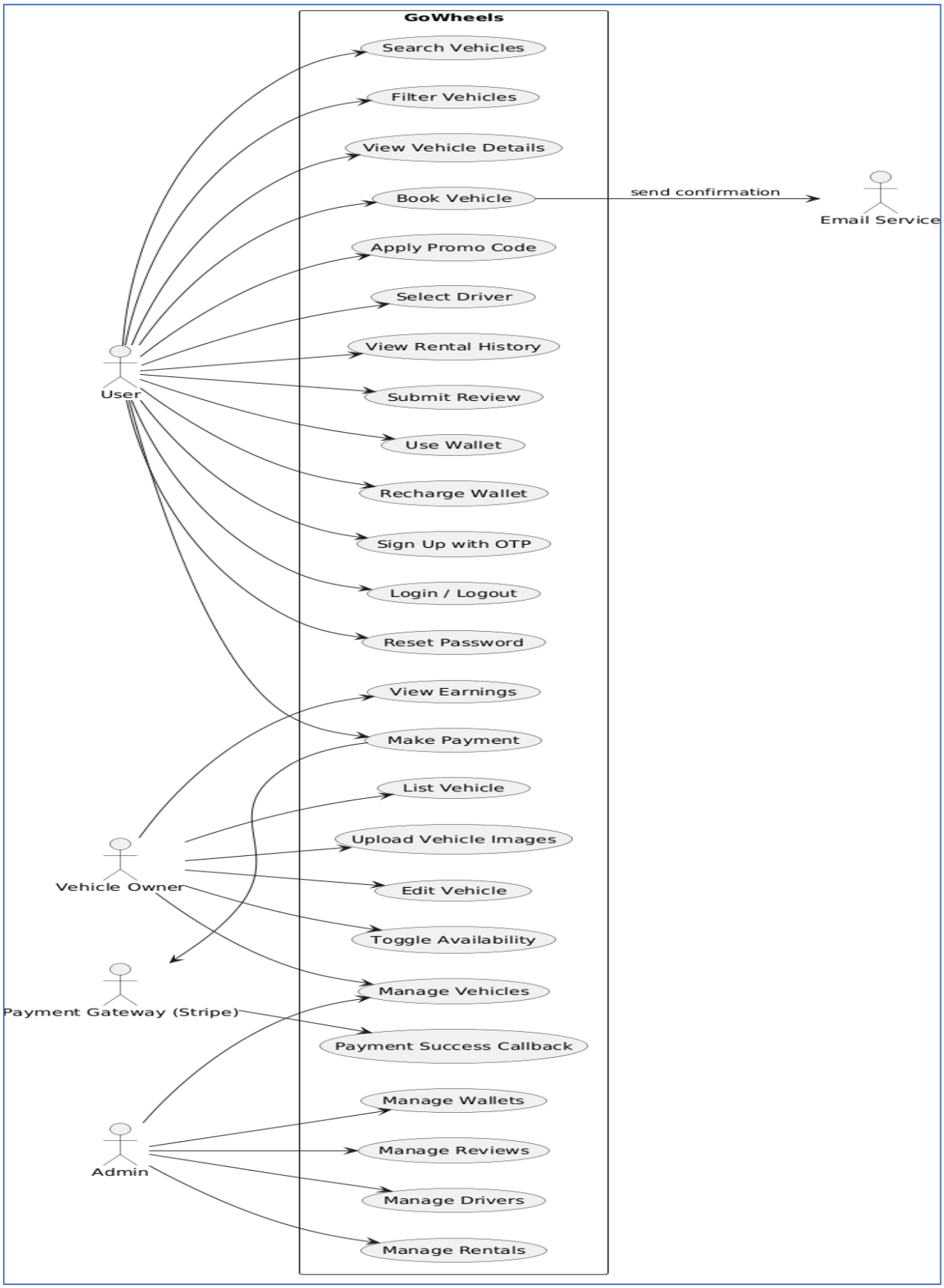

Fig. 5.3 Use Case Diagram

The Use Case Diagram of the GoWheels system illustrates the interaction between different actors and the system functionalities. It helps in understanding how users and external entities interact with the system to perform various operations.

Actors in the System:

1. **The main actors involved in the GoWheels system are:**
   i. **User (Customer):** The primary actor who uses the platform to search, book, and manage vehicle rentals.
   ii. **Vehicle Owner:** A user who lists and manages their vehicles for rental.
   iii. **Admin:** Responsible for managing the overall system including vehicles, drivers, rentals, and user activities.
   iv. **Payment Gateway (Stripe):** External system used for processing online payments securely.
   v. **Email Service:** External system used to send OTPs and booking confirmation emails.

2. **User Use Cases**

The User interacts with the system through several functionalities:

i. **Registration & Login:** Users can create an account using OTP verification and log in securely.
ii. **Search & Filter Vehicles:** Users can browse vehicles based on category, price, fuel type, and availability.
iii. **Book Vehicle:** Users can select rental dates, choose self-drive or driver option, and confirm booking.
iv. **Apply Promo Code:** Discounts can be applied during booking using valid promo codes.
v. **Make Payment:** Users can pay via wallet, online payment (Stripe), or cash on pickup.
vi. **View Rental History:** Users can track past, active, and upcoming bookings.
vii. **Submit Review:** After completing a rental, users can provide ratings and feedback.

3. **Vehicle Owner Use Cases**
   i. **List Vehicle:** Add new vehicles with details and images.
   ii. **Edit Vehicle:** Update vehicle information such as price, type, and location.
   iii. **Toggle Availability:** Enable or disable vehicle availability.
   iv. **View Earnings:** Monitor bookings and total earnings from rentals.

4. **Admin Use Cases**
   i. Manage Vehicles and Drivers
   ii. Monitor Rentals and Bookings
   iii. Manage Wallet Transactions
   iv. Review User Feedback

5. **External Interactions**
   i. The **Payment Gateway** is involved in processing online transactions securely.
   ii. The **Email Service** is used to send OTP verification and booking confirmation emails to users.

E. ER diagram

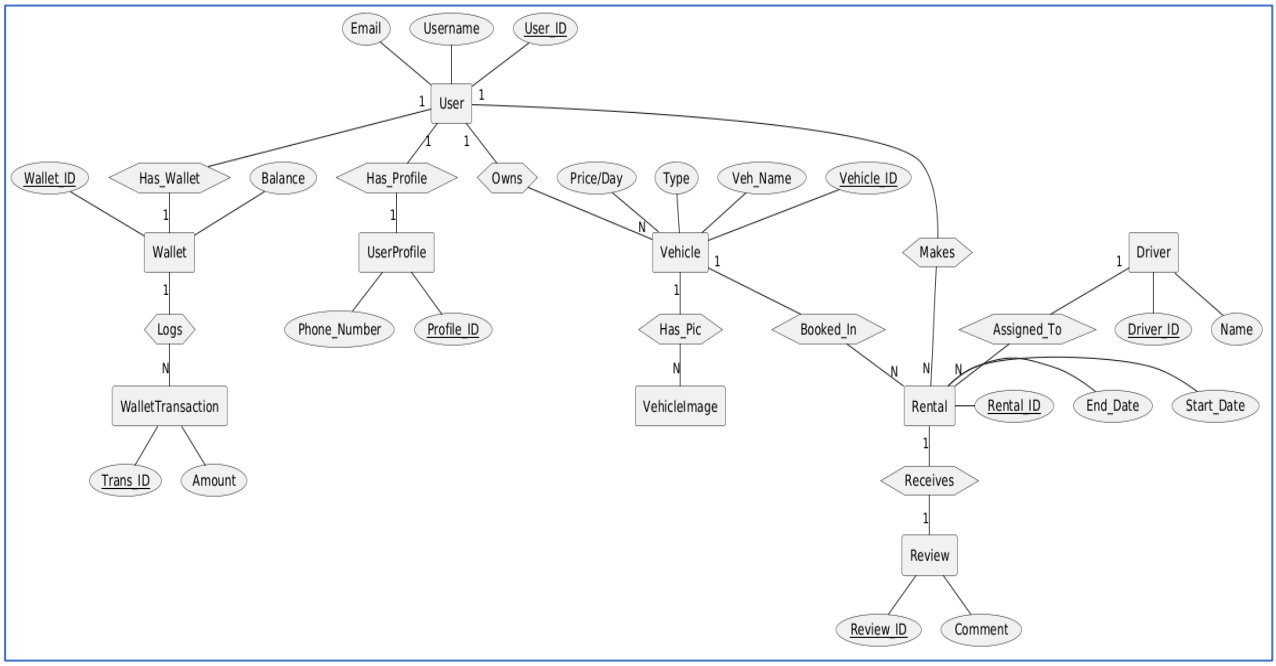

Fig. 5.4 ER Diagram

Table 7 Database table description

Entity Name	Description	Attributes
User	Represents a registered customer or vehicle owner	User_ID (PK), Username, Email, First_Name, Last_Name
UserProfile	Stores additional user information	Profile_ID (PK), Phone_Number
Wallet	Represents the user's digital wallet	Wallet_ID (PK), Balance, Updated_At
WalletTransaction	Stores wallet transaction history	Trans_ID (PK), Amount, Transaction_Type (Credit/Debit), Status, Date
Vehicle	Represents vehicles available for rent	Vehicle_ID (PK), Vehicle_Name, Type, Category, Price_Per_Day, Fuel_Type, Seats, Location, Availability
VehicleImage	Stores vehicle images	Image_ID (PK), Image_Path, Image_Type
Rental	Represents booking details	Rental_ID (PK), Start_Date, End_Date, Total_Price,

		Drive_Type, Payment_Mode, Promo_Code
Driver	Represents drivers available for hire	Driver_ID (PK), Name, Age, Experience, Rating, Price_Per_Day
Review	Stores feedback after rentals	Review_ID (PK), Cleanliness, Performance, Comfort, Driver_Rating, Comment

F. UML Class Diagram

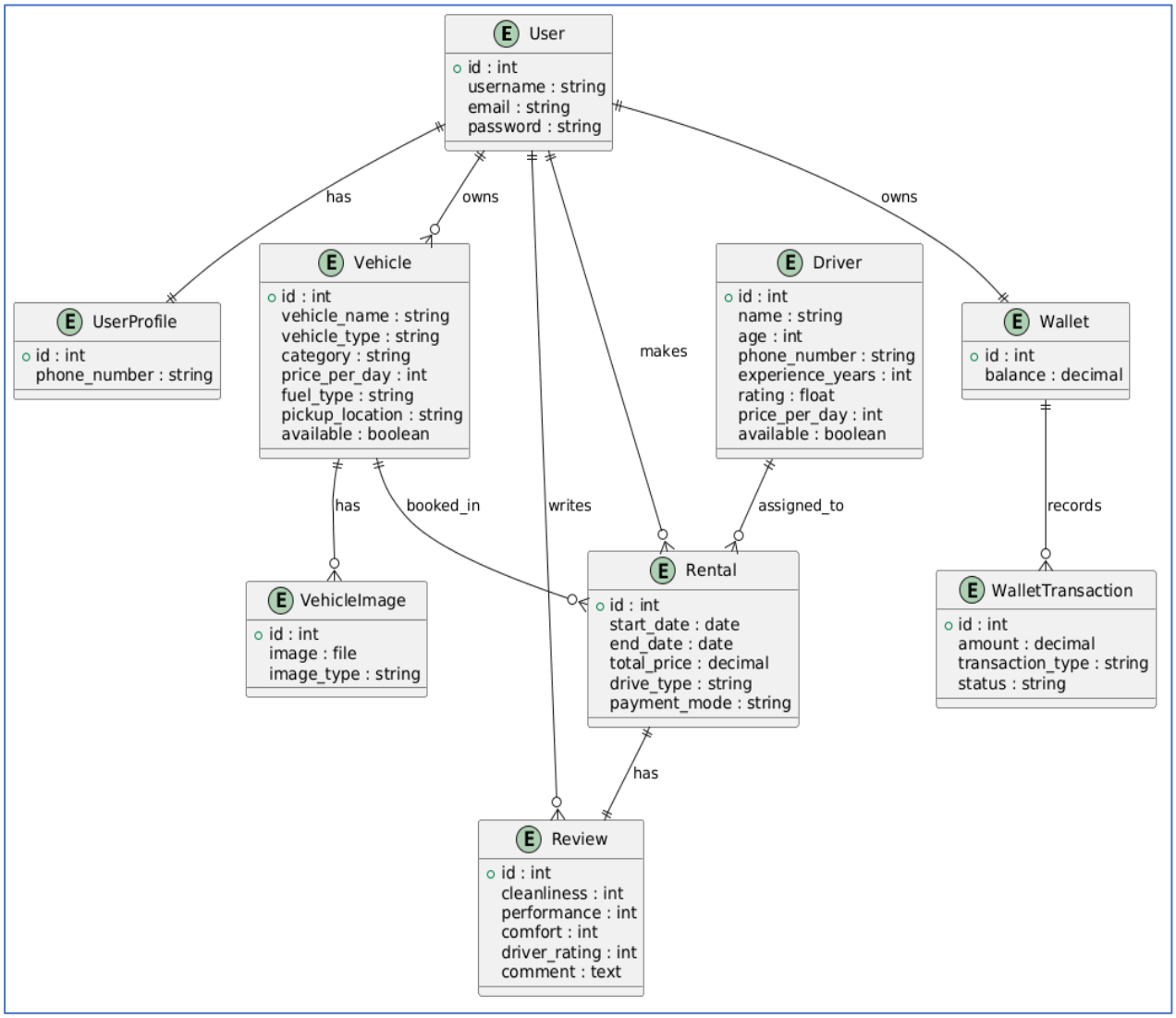

Fig. 5.5 UML Class Diagram

The Entity Relationship (ER) Diagram of the GoWheels system represents the structure of the database and the relationships between different entities involved in the vehicle rental platform. It helps in understanding how data is organized, stored, and interconnected within the system.

Main Entities

The system consists of the following key entities:

i. **User:**
Stores user details such as username, email, and password. A user can act as both a customer and a vehicle owner.

ii. **User_Profile:**
Contains additional information about the user, such as phone number. It has a one-to-one relationship with the User entity.

iii. **Vehicle:**
Stores details of vehicles listed for rent, including vehicle name, type, category, price per day, fuel type, and availability status.

iv. **Vehicle_Image:**
Stores images related to vehicles (front, back, inside, etc.). A vehicle can have multiple images.

v. **Driver:**
Contains information about drivers such as name, age, experience, rating, and availability. Drivers can be assigned to rentals.

vi. **Rental:**
This is the central entity of the system. It stores booking details such as rental dates, total price, payment mode, and drive type. It connects users, vehicles, and drivers.

vii. **Wallet:**
Maintains the balance of each user for making payments within the system.

viii. **Wallet_Transaction:**
Stores transaction records including credit and debit operations performed on the wallet.

ix. **Review:**
Stores user feedback including ratings for cleanliness, performance, comfort, and optional driver rating.

Relationships Between Entities

The relationships between entities are defined as follows:

i. **User – UserProfile:**
   One-to-One relationship. Each user has exactly one profile.
ii. **User – Wallet:**
   One-to-One relationship. Each user has one wallet associated with their account.
iii. **User – Vehicle:**
   One-to-Many relationship. A user (owner) can list multiple vehicles.
iv. **Vehicle – VehicleImage:**
   One-to-Many relationship. A vehicle can have multiple images.
v. **User – Rental:**
   One-to-Many relationship. A user can make multiple bookings.
vi. **Vehicle – Rental:**
   One-to-Many relationship. A vehicle can be booked multiple times.
vii. **Driver – Rental:**
   One-to-Many relationship. A driver can be assigned to multiple rentals (optional relationship).
viii. **Rental – Review:**
   One-to-One relationship. Each completed rental can have one review.
ix. **Wallet – WalletTransaction:**
   One-to-Many relationship. A wallet can have multiple transactions.
x. **User – Review:**
   One-to-Many relationship. A user can write multiple reviews.

G. Site map

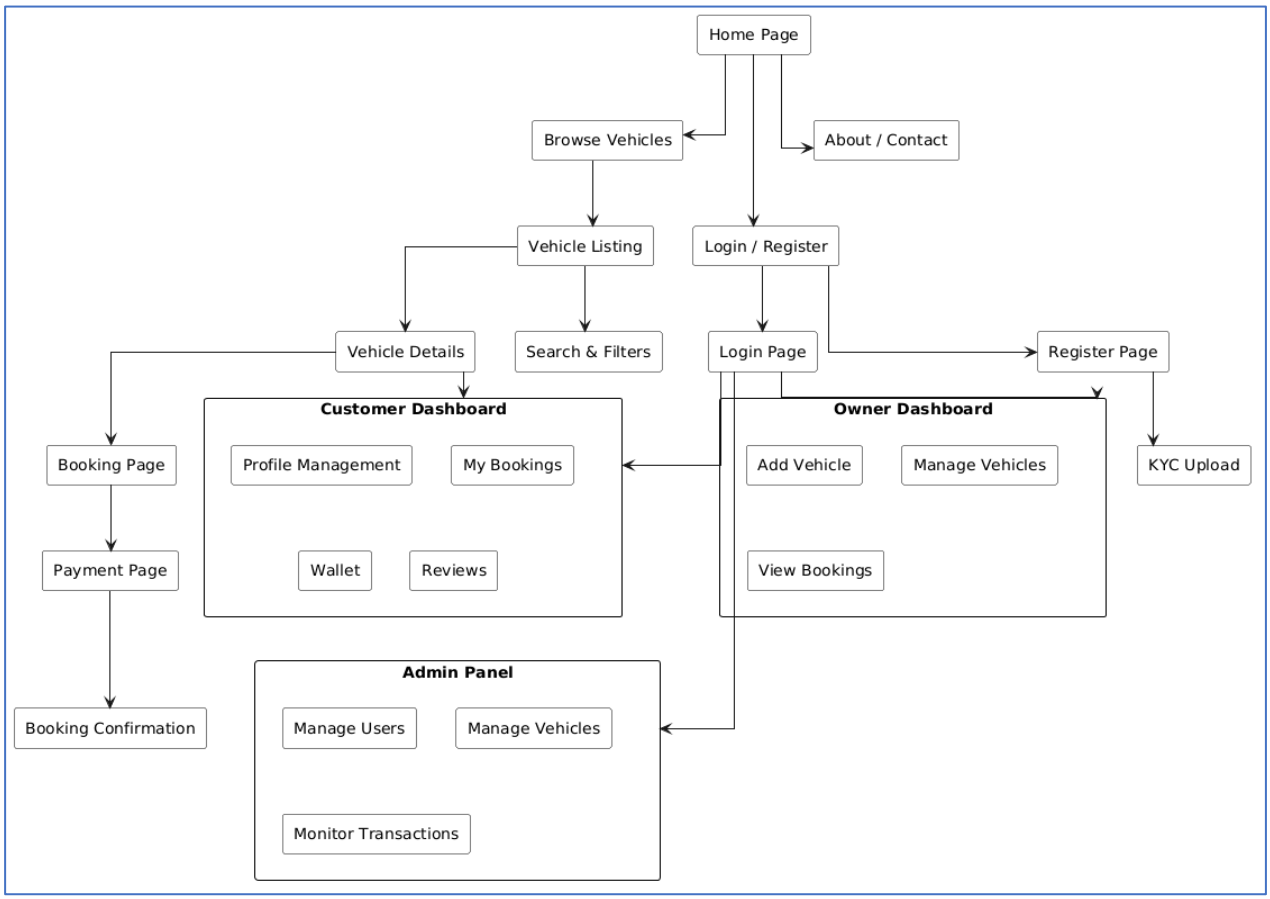

Fig. 5.6 Site map

The site map of the GoWheels system represents the overall structure and navigation flow of the web application. It organizes all major modules and pages in a hierarchical manner, making it easier for users to navigate and access different features of the platform.

At the top level, the **Home page** acts as the central hub of the system. From here, users can access all primary sections including Authentication, Vehicles, User Dashboard, Owner Panel, and additional support pages.

The **Authentication module** includes pages such as Login, Signup, Forgot Password, and Reset Password. This module ensures secure access to the system and allows users to create accounts, log in, and recover forgotten credentials using OTP verification.

The **Vehicles section** is one of the core components of the system. It allows users to browse available vehicles, apply search and filter options, view detailed vehicle information, and proceed to rent a vehicle. This section connects directly to the booking functionality of the system.

The **User Dashboard** provides personalized features for logged-in users. It includes Rent History to track past and current bookings, Submit Review for feedback on vehicles and drivers, and Payments/Wallet for managing transactions and balances.

The **Owner Panel** is designed for users who list their vehicles on the platform. It includes features such as List Vehicle, Your Vehicles (to manage owned vehicles), Edit Vehicle details, and Toggle Availability to control whether a vehicle is available for booking.

Additionally, the system includes an **Extras section**, which contains pages like Helpdesk for user support and Offers for promotional deals and discounts.

Chapter 6

Results and Applications

6.1 Introduction to Implemented System

This chapter demonstrates the final graphical user interfaces (GUI) and functional modules of the "GoWheels" web application. The system has been successfully developed using the Python Django framework and deployed on a local development server for testing. The following sections showcase the core features, user workflows, and database management interfaces designed to resolve the limitations of traditional vehicle rental systems.

6.2 Public Interfaces and Authentication

The initial interaction with the GoWheels platform is designed to be welcoming, informative, and secure.

A. Home Page

The Home Page serves as the primary landing page, featuring a modern, responsive design. It introduces the platform's core services, highlights available vehicle categories, and provides intuitive navigation to the booking engine and helpdesk.

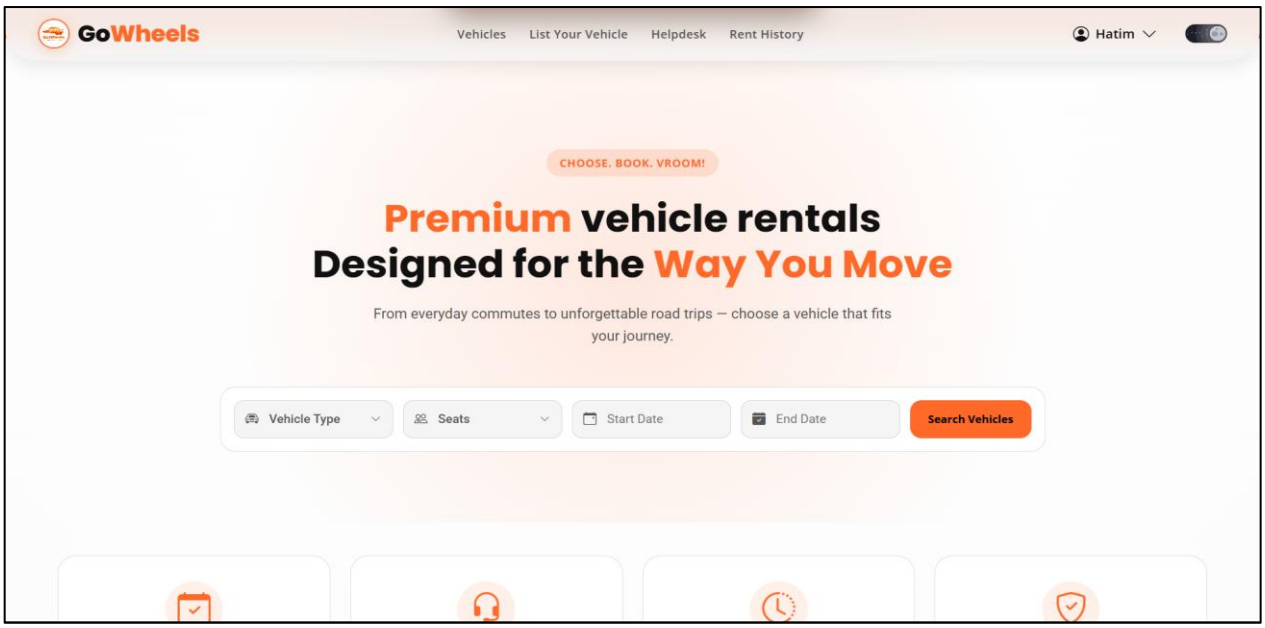

Fig 6.1:Home page

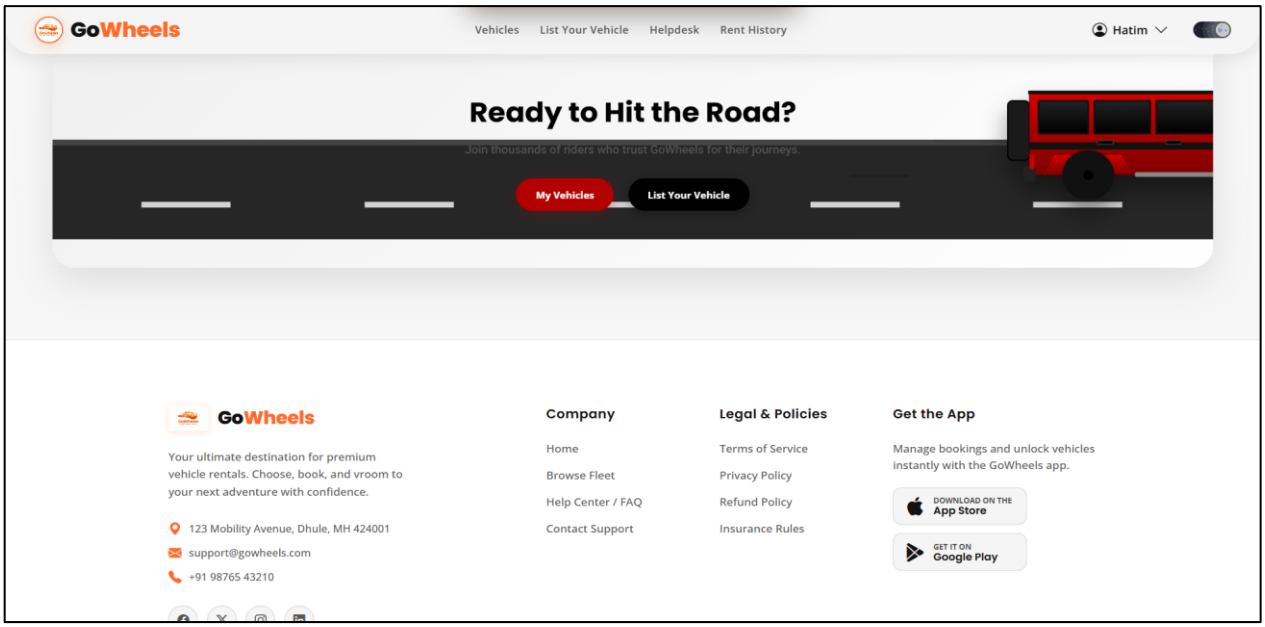

Fig 6.2 :Footer and abouts

B. User Registration (Sign Up) with OTP Verification

To ensure security and prevent spam accounts, the registration process requires users to provide basic details and verify their email address via a real-time OTP (One-Time Password) system.

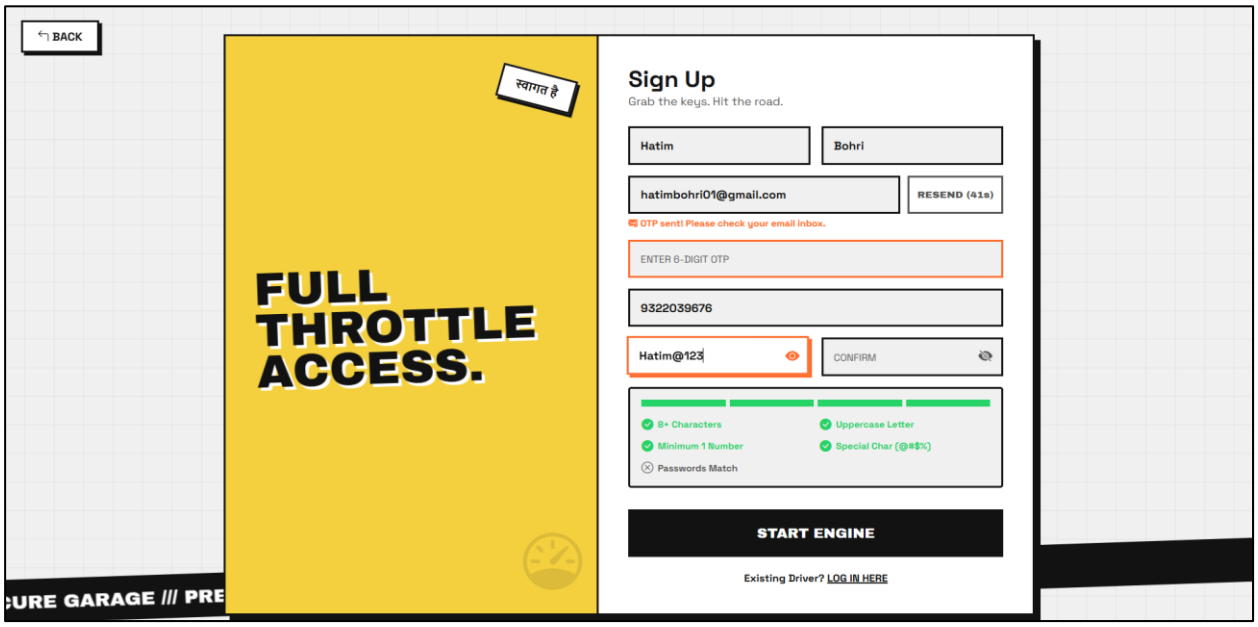

Fig 6.3: Sign Up page

C. Secure Login Interface

Registered users and vehicle host can securely log into their accounts. The system includes rate-limiting features to prevent brute-force attacks after consecutive failed attempts.

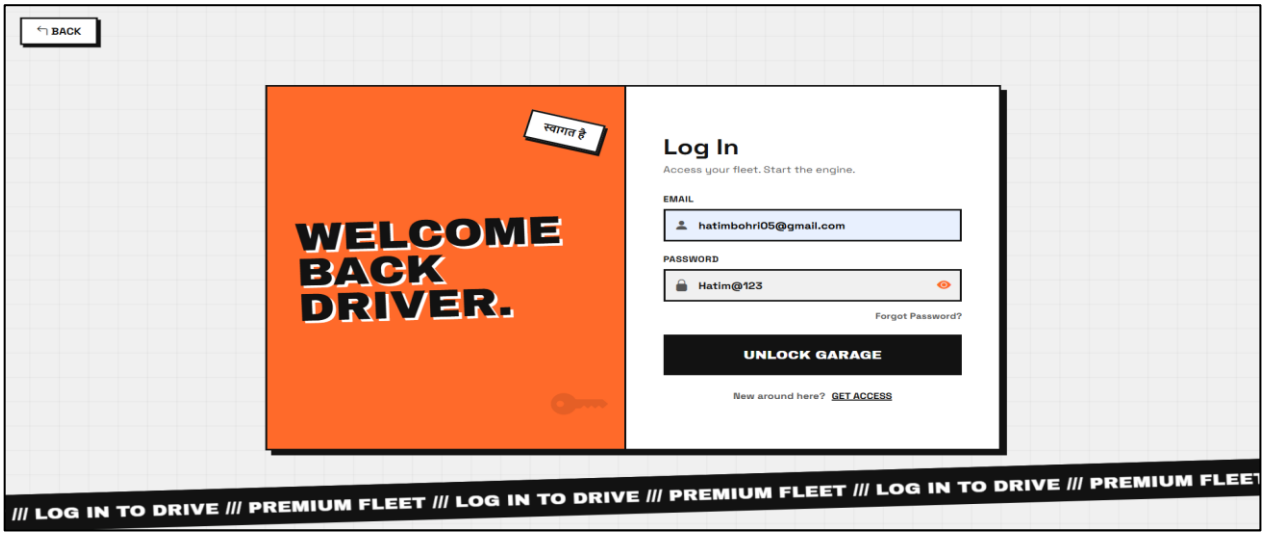

Fig 6.4: Log In page

6.3 Vehicle Discovery and Booking Engine

A. Vehicle Browsing and Advanced Search Filter

The Vehicles page displays a dynamic catalog of all available vehicles. Users can utilize the multi-column search and filter sidebar to sort vehicles by category, seating capacity, price range, and availability dates and allowing users to browse, filter, and book vehicles seamlessly.

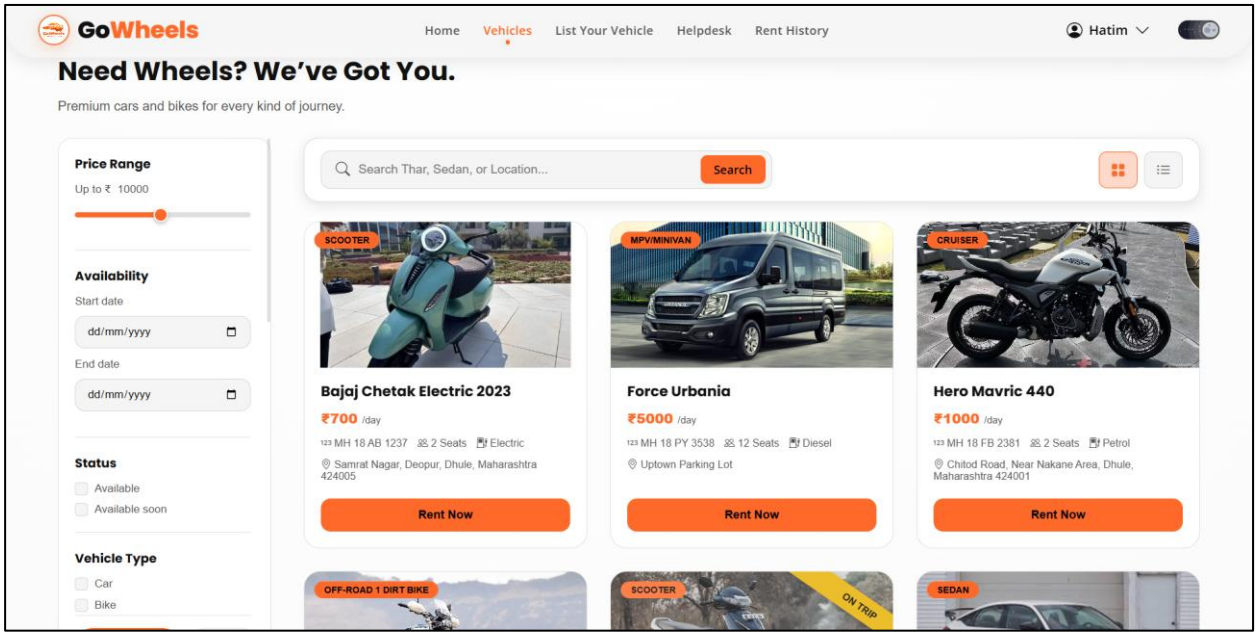

Fig 6.5: Browse Vehicles

B. Vehicle Details and Rental Configuration

When a user selects a vehicle, they are presented with detailed specifications and images. On this page, users configure their rental by selecting start and end dates, and choosing their preferred drive mode ("Self-Drive" or "With GoWheels Driver"). The platform provides real-time pricing updates, instantly reflecting any changes in the rental duration or the addition o. Furthermore, users can view transparent terms and conditions, mileage limits, and immediate availability status to ensure a seamless decision-making process before proceeding to checkout.

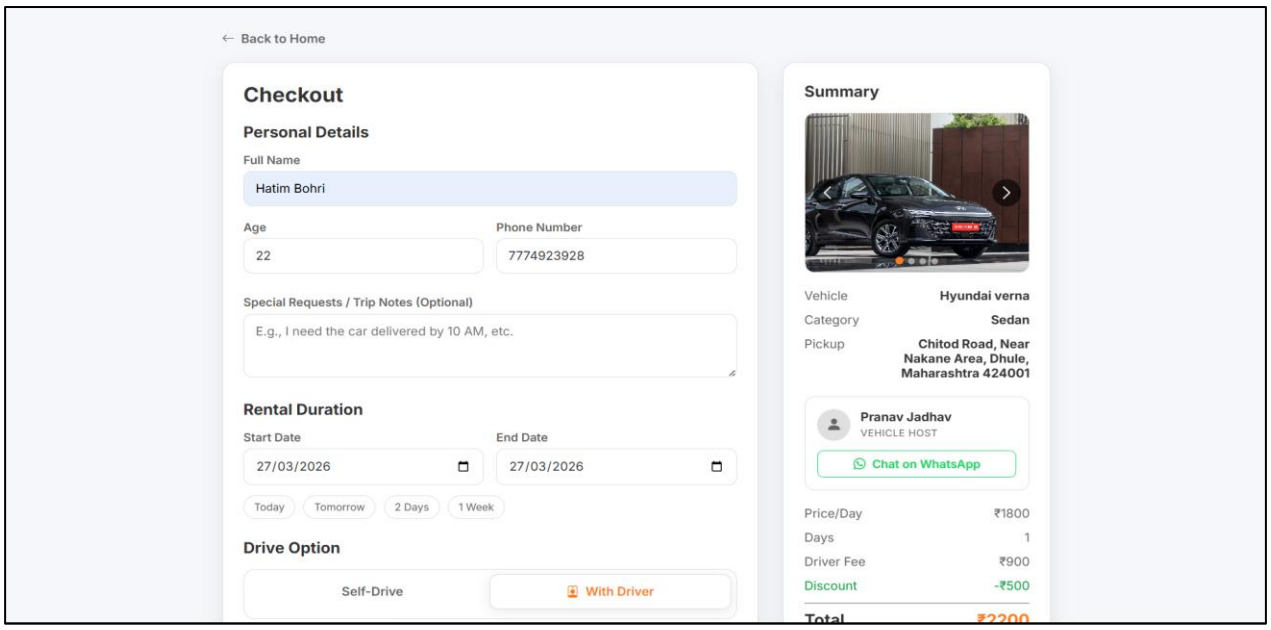

Fig 6.6: Rent Vehicle page

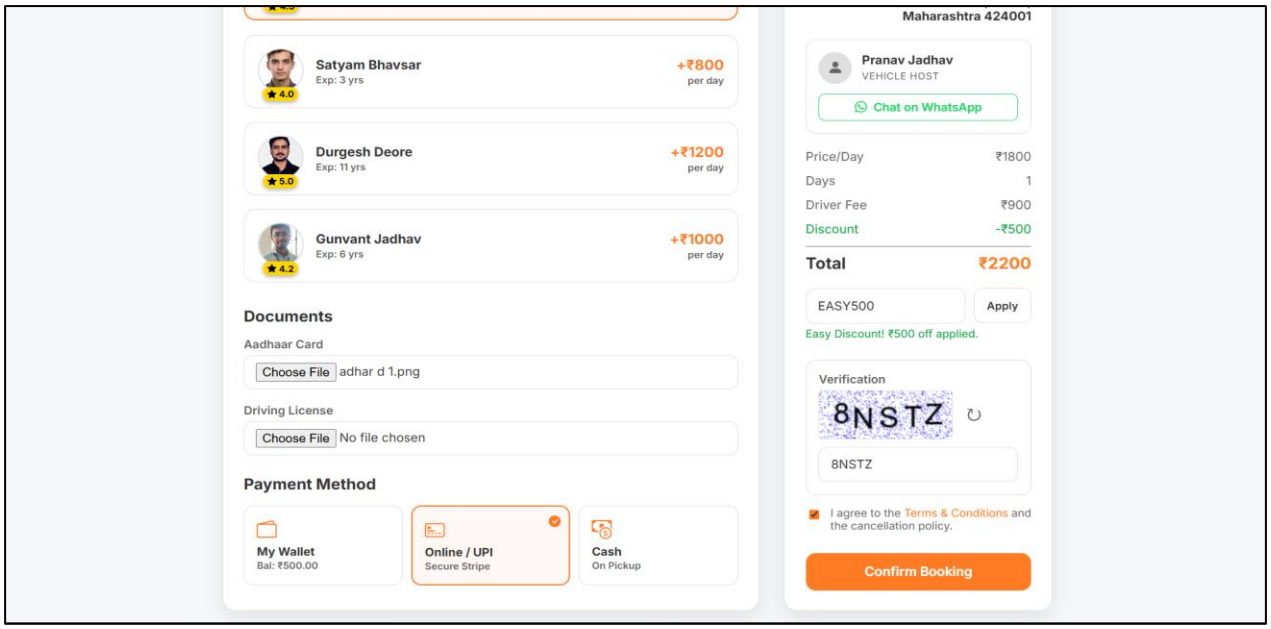

Fig 6.7: Rent Vehicle page with Driver selection and payment mode selection

C. KYC Document Upload

To maintain trust and security, users must upload digital copies of their Aadhaar Card and valid Driving License before proceeding to checkout.

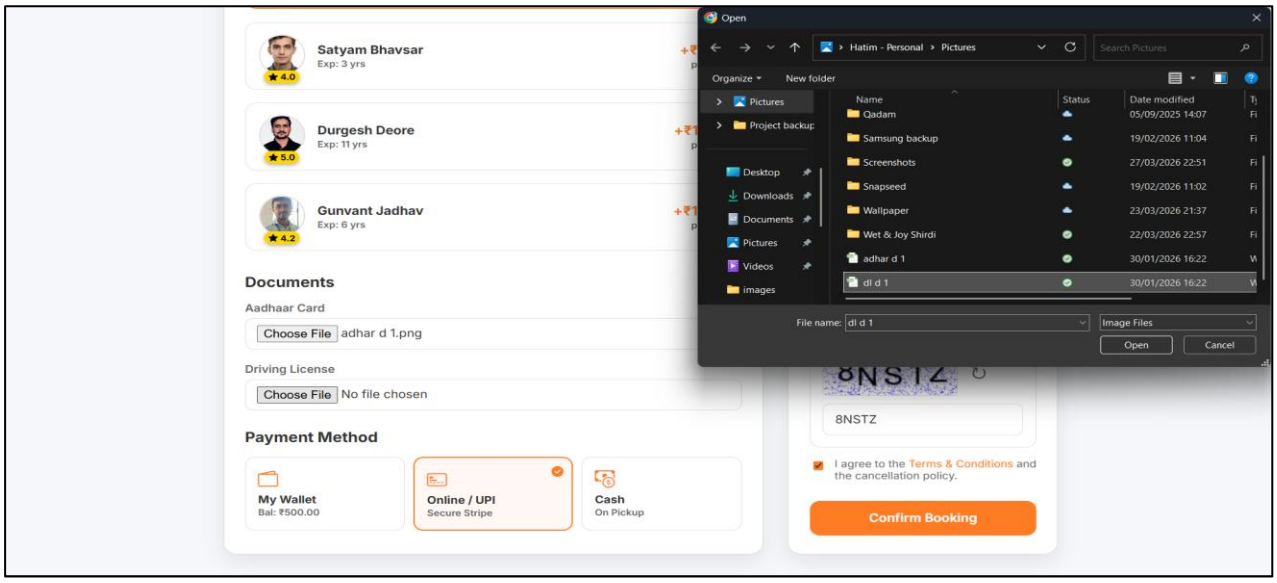

Fig 6.8: Documents upload

6.4 Payment Integration and Wallet System

A. Digital Wallet Dashboard

Users have access to an integrated digital wallet. This page displays their current balance and a detailed ledger of all past transactions (Credits and Debits).

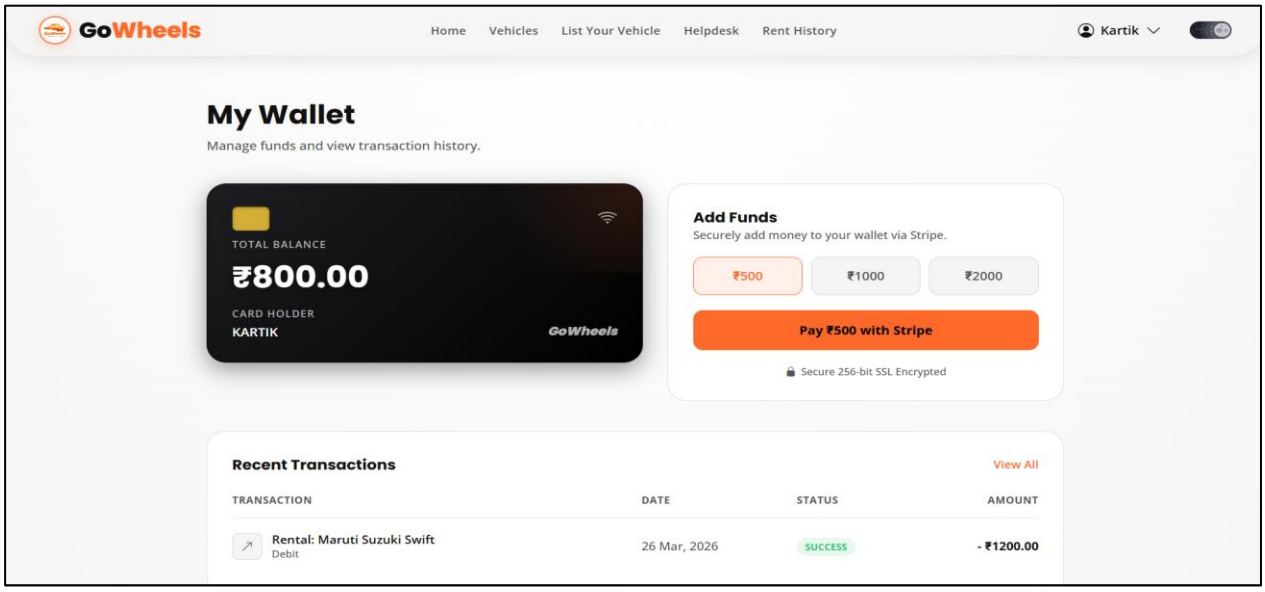

Fig 6.9: My wallet page

B. Payment Checkout (Stripe Integration)

For users paying online or recharging their wallet, the system securely redirects them to the Stripe Payment Gateway to process card transactions without storing sensitive financial data on local servers.

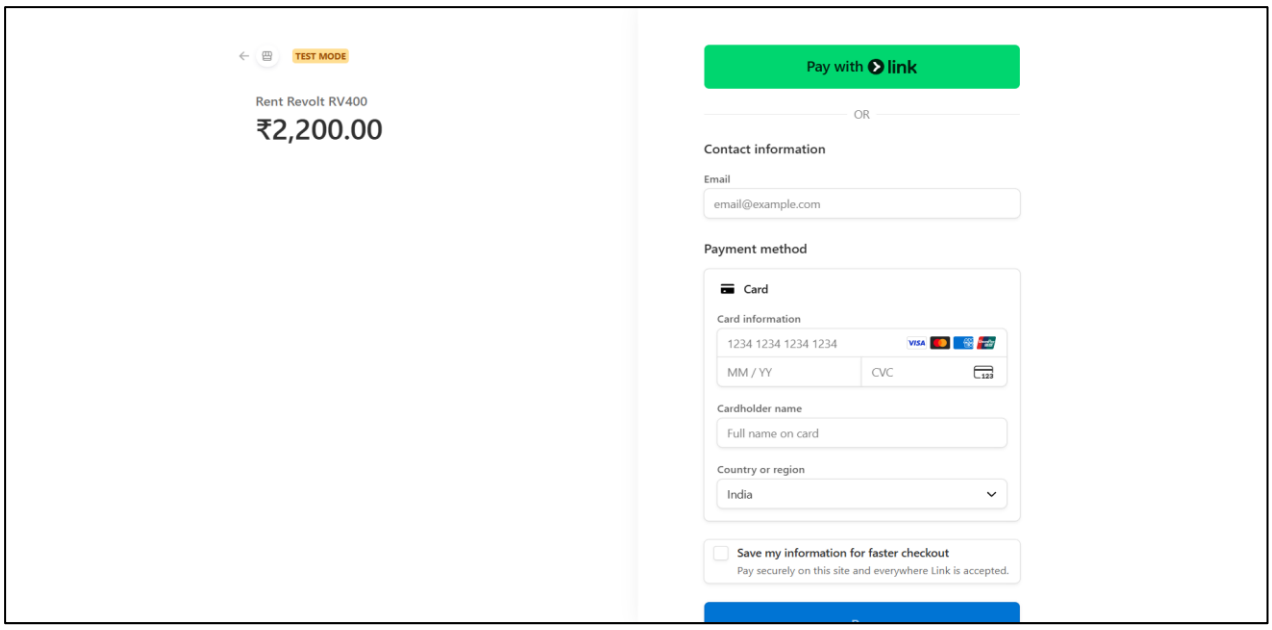

Fig 6.10: Stripe payment gateway

C. Dynamic QR-Based UPI Payment System

The system provides a seamless **UPI-based payment option** by generating a **lifetime dynamic QR code** for each transaction session. This QR code is designed to automatically reflect **real-time bill updates**, ensuring accuracy and flexibility during checkout.

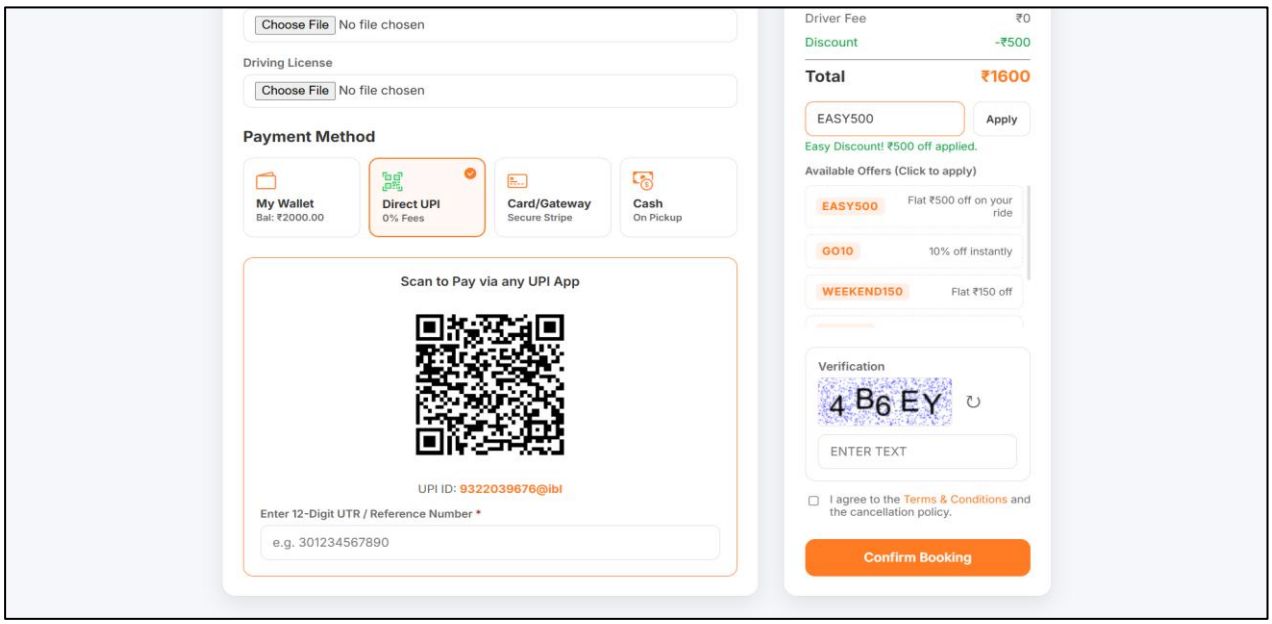

Fig 6.11: UPI payment gateway

6.5 User Dashboard and Rental History

After completing a booking, users can track their trips and manage their profile.

A. Rental History and Analytics

The Rent History dashboard provides users with a comprehensive overview of their active, upcoming, and completed rentals. It includes visual graphs (spending analytics) and tracks their loyalty tier (e.g., Gold, Platinum) based on total expenditure. Additionally, this section offers quick-action buttons to leave feedback on rides.

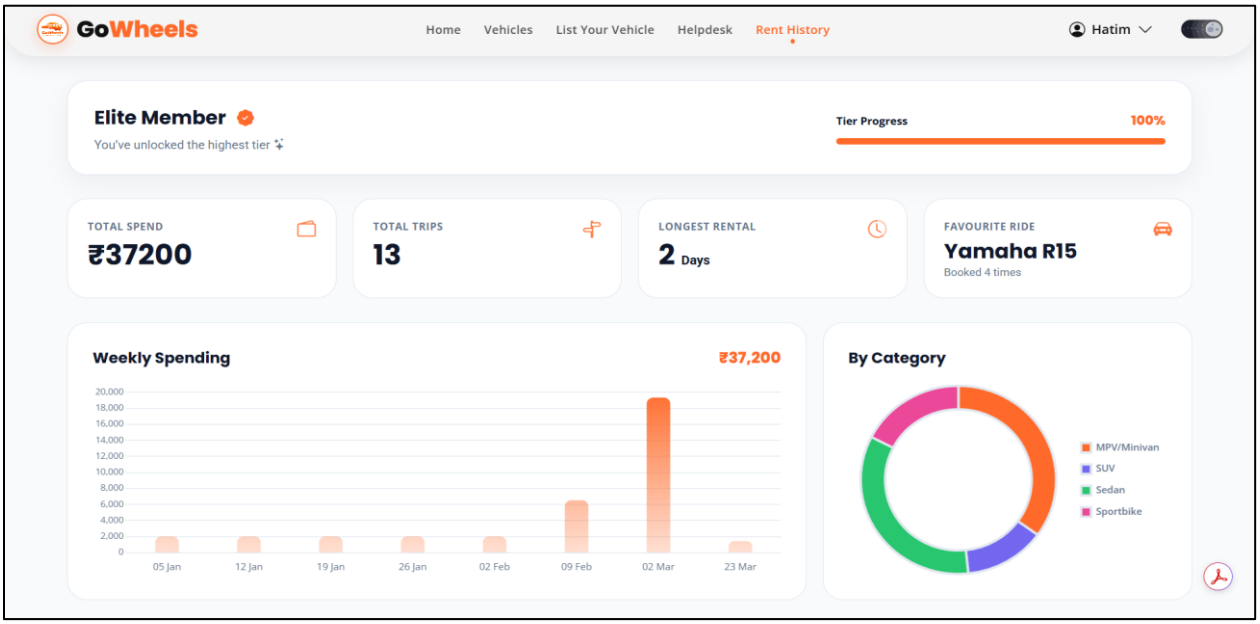

Fig 6.12: Spending analytics

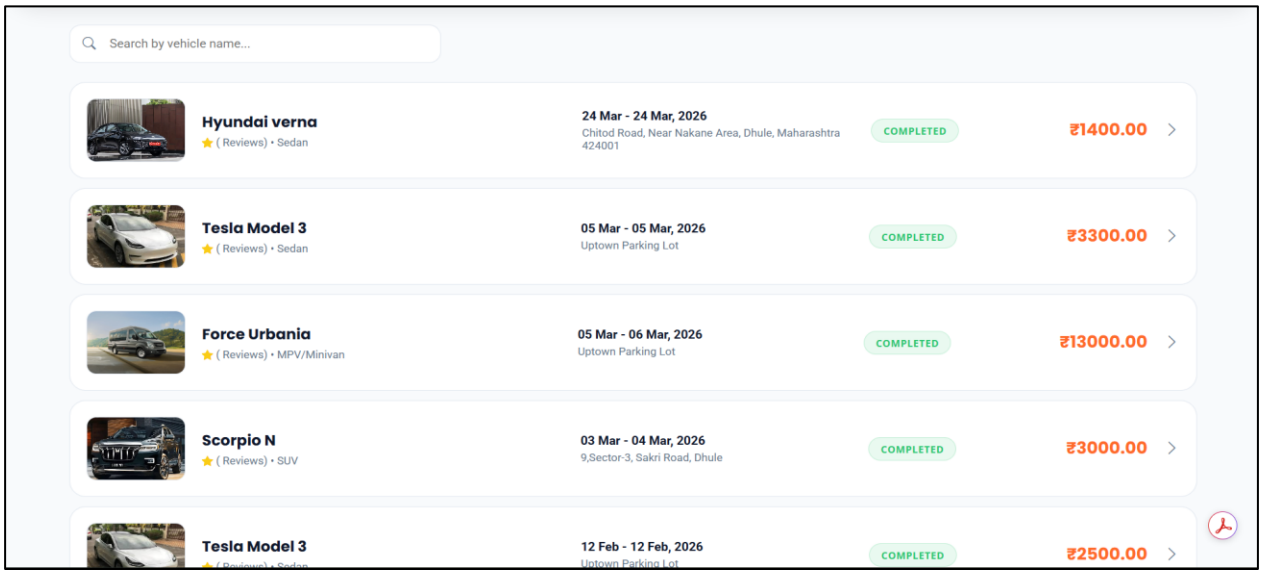

Fig 6.13: Trip summary and history

B. Premium Email Invoice

Upon successful booking, the system automatically generates and dispatches a detailed, premium HTML invoice to the user's registered email, outlining the trip itinerary, payment breakdown, and pickup guidelines. This communication also features embedded quick links that allow users to easily modify or cancel their GoWheels reservation directly from their inbox. Furthermore, the invoice provides direct contacts for 24/7 roadside assistance and includes a downloadable calendar attachment to help keep their travel schedule organized.

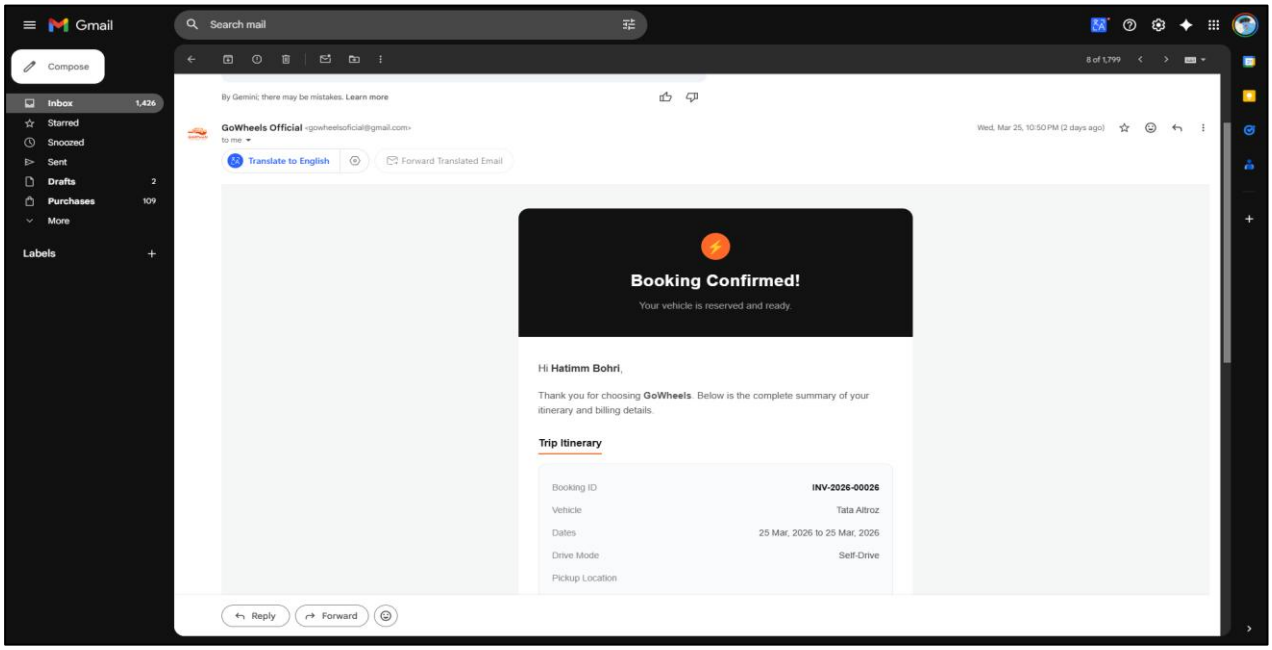

Fig 6.14: Booking Confirmation mail

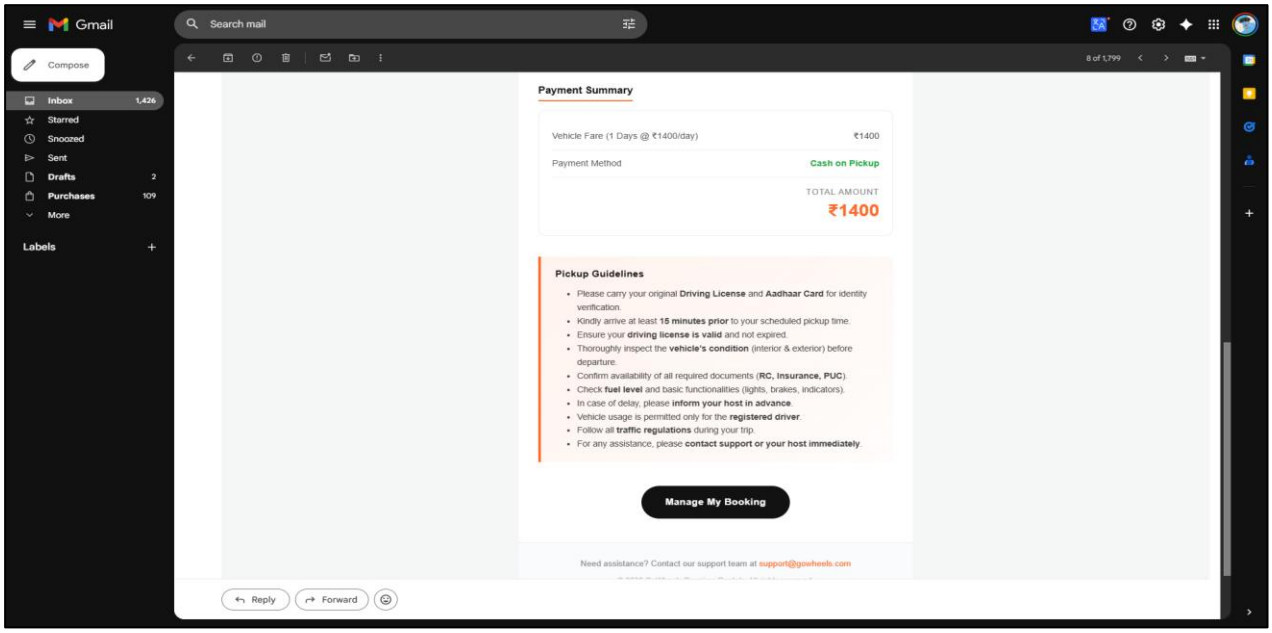

Fig 6.15: Bills and Guidelines

C. Post-Trip Review System

Once a trip is marked as completed, users are prompted to submit a review, rating the vehicle's cleanliness, performance, comfort, and the driver's behaviour (if applicable).

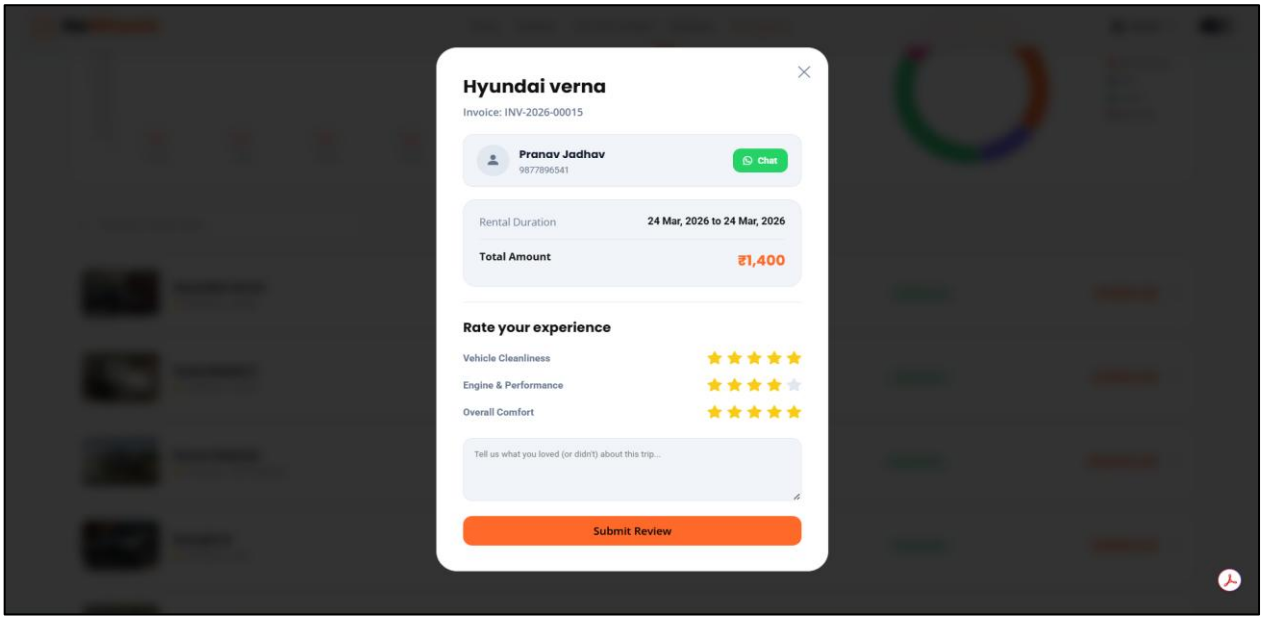

Fig 6.16: Review trip

6.6 Vehicle Owner Module

GoWheels operates as a dual-sided marketplace, providing dedicated tools for vehicle host to monetize their assets.

A. Add New Vehicle

Host can easily list a new vehicle by filling out specifications and uploading multiple images (front, back, interior). The streamlined form captures essential basic details like the desired daily rental price, owner contact information, and specific vehicle attributes such as fuel type, seating capacity, and category. To maximize booking potential, the platform encourages hosts to provide high-quality, well-lit photos and define convenient pickup locations, allowing users to turn their idle vehicles into an active earning stream in just a few minutes.

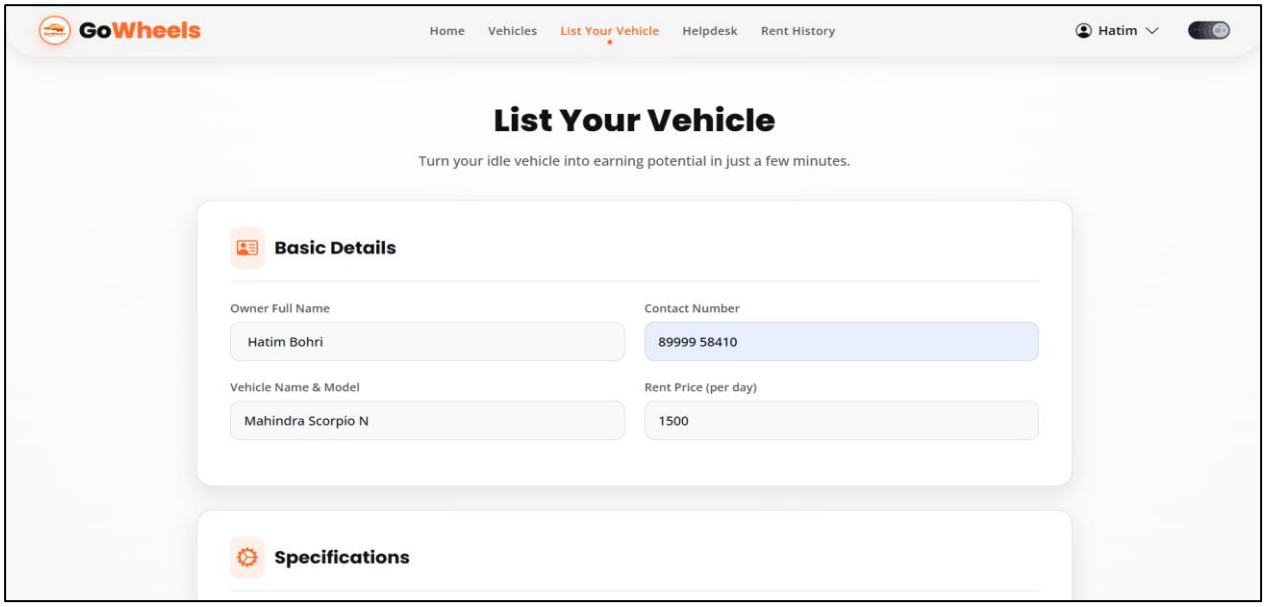

Fig 6.17: List Vehicle

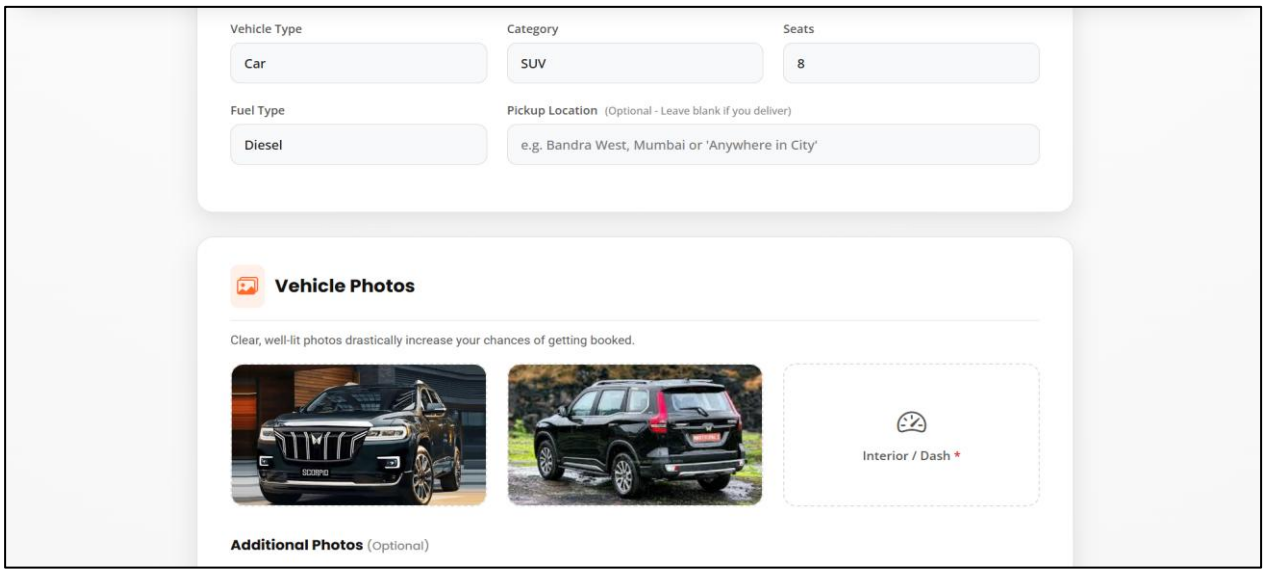

Fig 6.18: Upload Vehicle images and details

B. Owner Inventory Dashboard

Hosts have a specialized dashboard to view all their listed vehicles, track total earnings, and instantly toggle a vehicle's availability status (Active/Inactive) using a dynamic switch. This centralized hub provides at-a-glance performance metrics, summarizing the total fleet size, vehicles currently on rent, and overall revenue generated. To streamline inventory management, host can easily filter their listings by vehicle type or current availability, and use the integrated search bar to locate specific assets quickly.

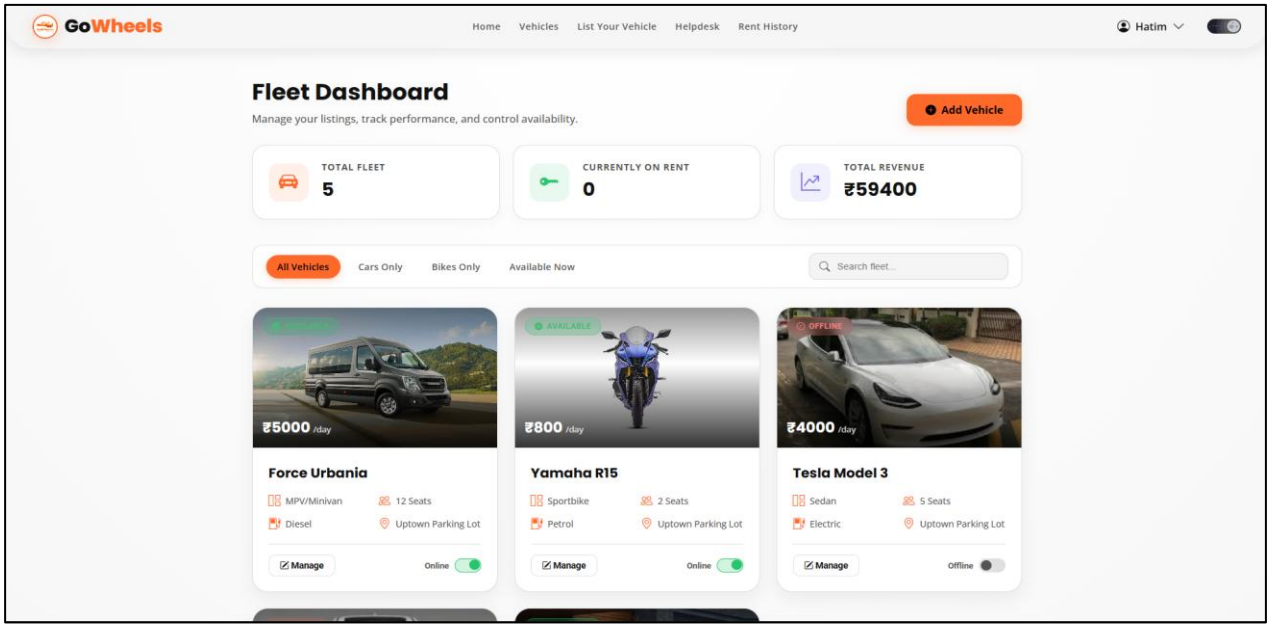

Fig 6.19: Host Dashboard

C. Edit vehicle details

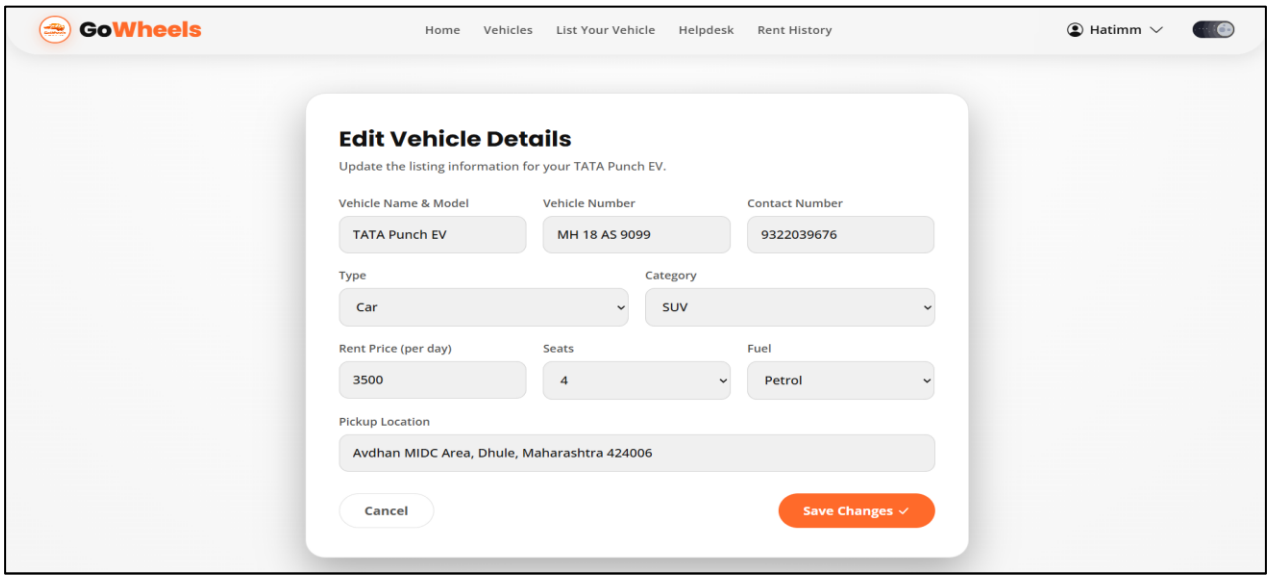

Fig 6.20: Edit Vehicle details page

6.7 Administrator Panel

A. Centralized Management

The admin panel provides full CRUD (Create, Read, Update, Delete) access to the database, allowing administrators to verify KYC documents, manage driver profiles, and resolve transaction disputes. It features a real-time "Recent actions" log that meticulously tracks all administrative changes, such as user modifications or wallet adjustments, ensuring complete system accountability. Furthermore, administrators can comprehensively oversee authentication protocols by seamlessly managing user roles, groups, and permissions directly from the main dashboard.

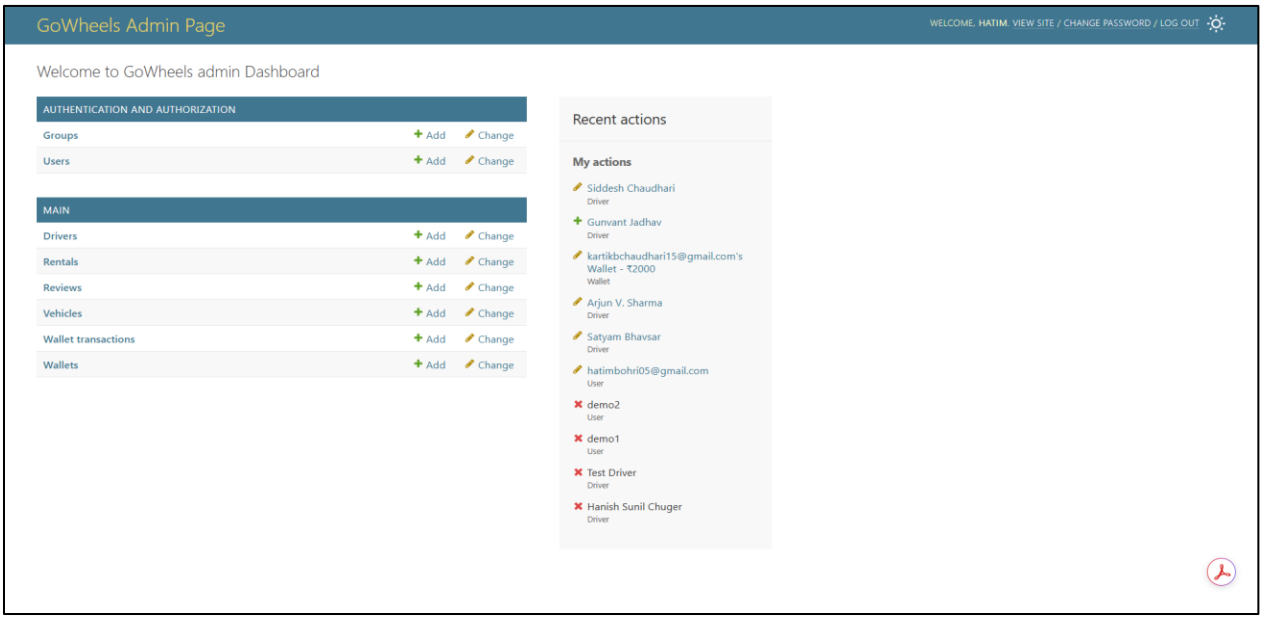

Fig 6.21: GoWheels Admin page

B. Driver Management

Through the dedicated Drivers module, administrators can easily onboard new chauffeurs or update existing driver profiles. This section allows the admin team to review driver credentials, monitor overall performance statuses, and ensure that all personnel consistently meet the platform's safety and quality standards before being assigned to customer trips.

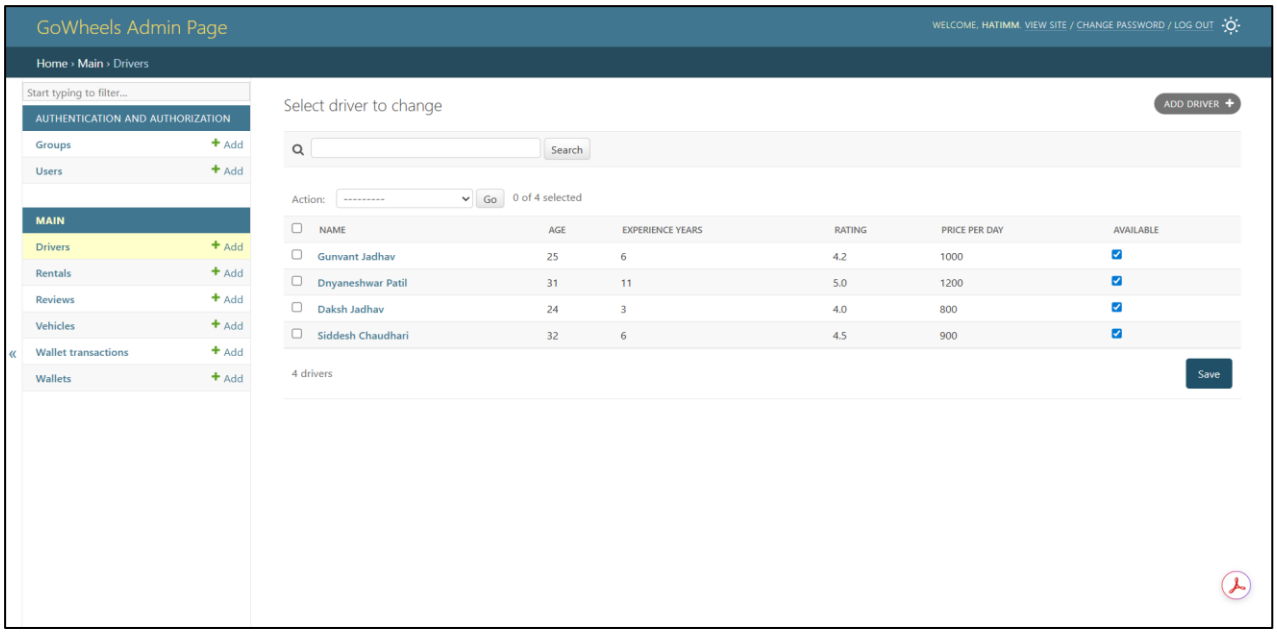

Fig 6.22: Driver dashboard

C. Review Moderation

The Reviews section empowers administrators to monitor user feedback and safeguard the platform's reputation. Admins can read, moderate, and address customer reviews left for both vehicles and drivers, ensuring that any service issues are promptly handled and that high user-satisfaction standards are maintained across the ecosystem.

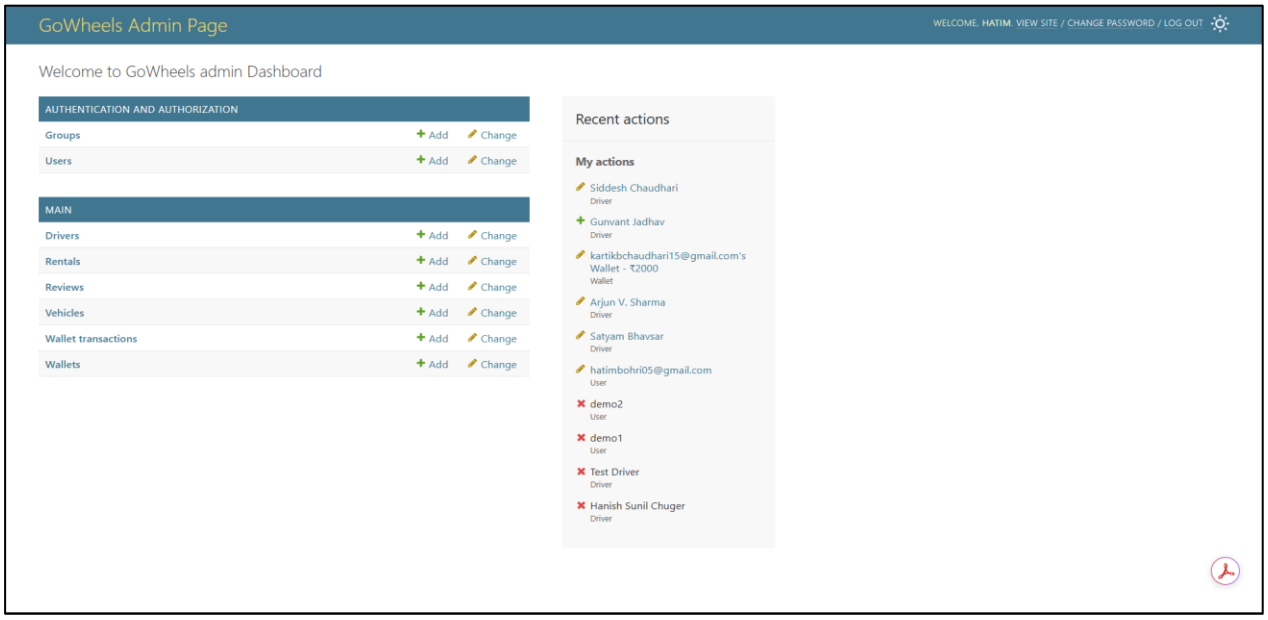

Fig 6.23: Reviews dashboard

D. Vehicle Fleet Oversight

The Vehicles table grants administrators total control over the platform's comprehensive vehicle inventory. From this centralized hub, the admin can quickly approve new vehicle listings submitted by host, update specific car details, and instantly remove or suspend listings that no longer comply with the platform's strict operational guidelines.

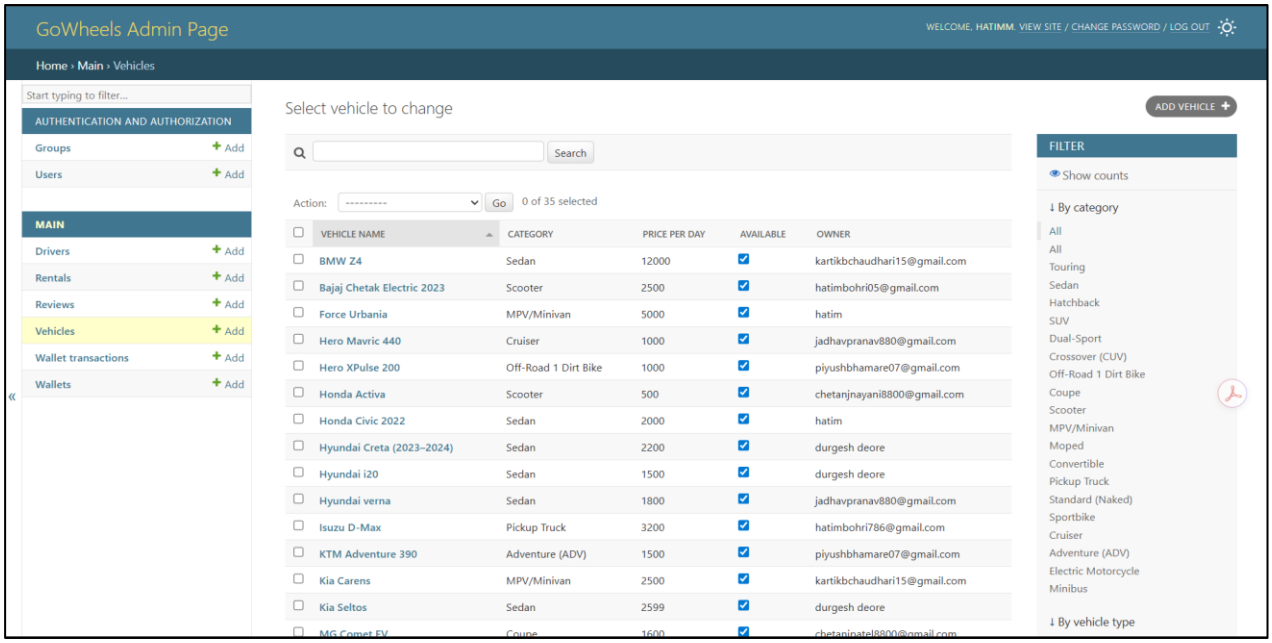

Fig 6.24: Vehicles dashboard

F. Wallet Transactions The Wallet Transactions module provides administrators with a transparent, chronological ledger of all financial movements within the platform's digital wallet ecosystem. Admins can effortlessly track, review, and audit individual user top-ups, rental deductions, and vehicle owner payouts to ensure absolute financial accuracy. This centralized oversight empowers the administrative team to swiftly investigate payment disputes, authorize manual wallet adjustments, and process seamless refunds when necessary.

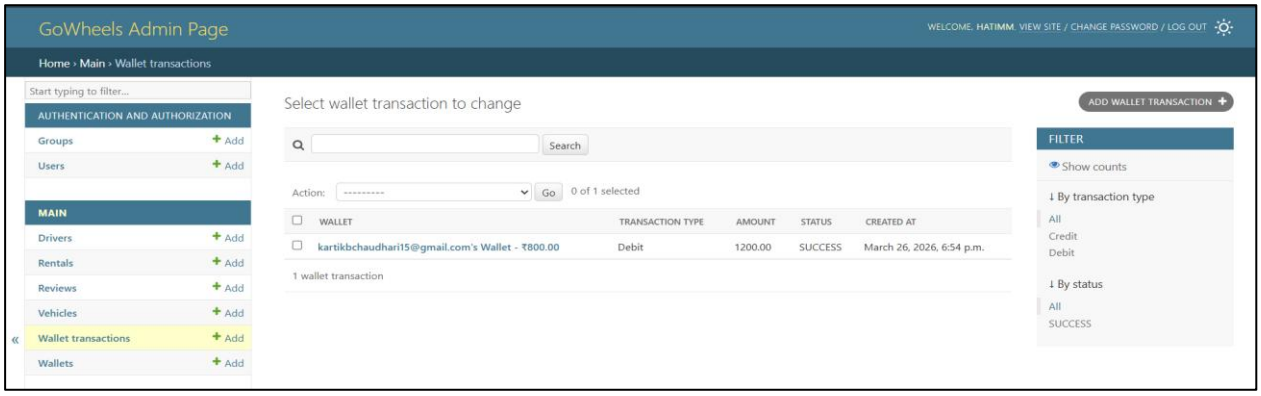

Fig 6.25: Wallet transaction dashboard

6.8 Customer Engagement and Support Modules

To ensure high user retention and provide a seamless experience, the GoWheels platform incorporates dedicated modules for customer support and financial benefits.

A. Promotional Offers and Discounts

To encourage platform usage and provide cost-effective travel solutions, GoWheels features a dedicated "Offers" page. This interface elegantly displays currently active promotional codes (such as *FIRST200* for new users, *WEEKEND150*, and *VIP25* for premium bookings). Users can browse this page to find relevant coupon codes and apply them directly within the booking engine to instantly recalculate their final rental price.

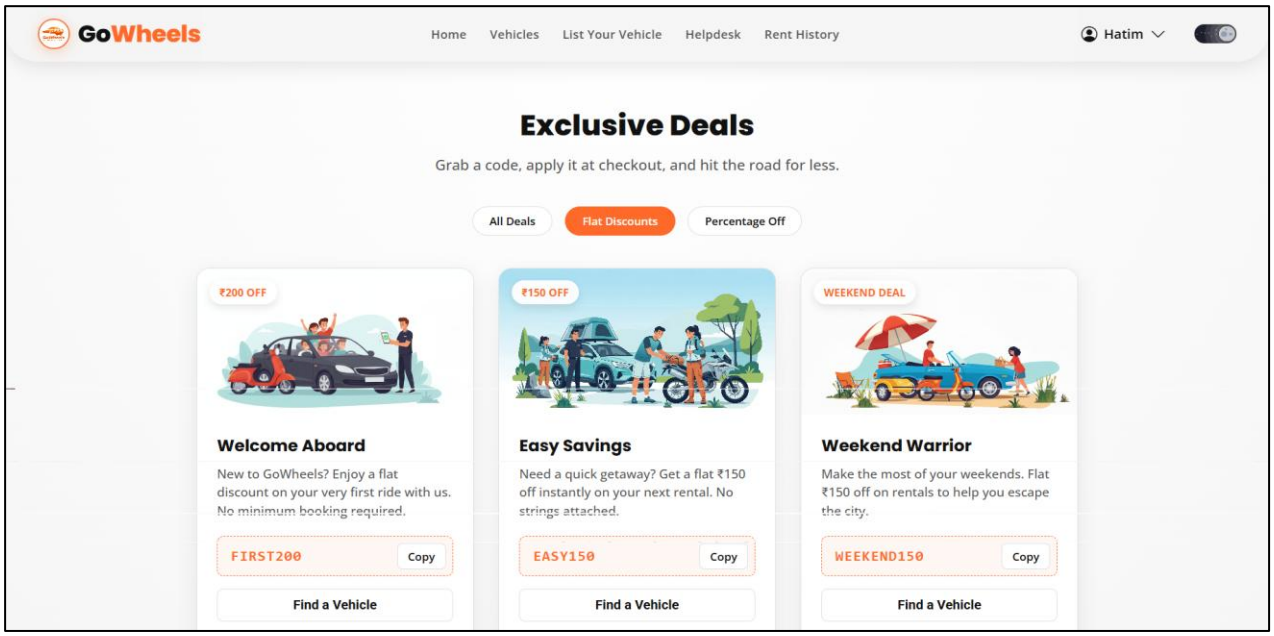

Fig 6.26: Offers page

B. Helpdesk and User Support Interface

To assist users with any queries regarding the rental lifecycle, KYC document verification, or wallet transactions, a comprehensive Helpdesk page is integrated into the system. It serves as a centralized support hub featuring standard guidelines, Frequently Asked Questions (FAQs), and direct contact information (e.g., support email and helpline). This ensures that both vehicle owners and renters have immediate access to assistance, minimizing operational friction and building trust in the platform.

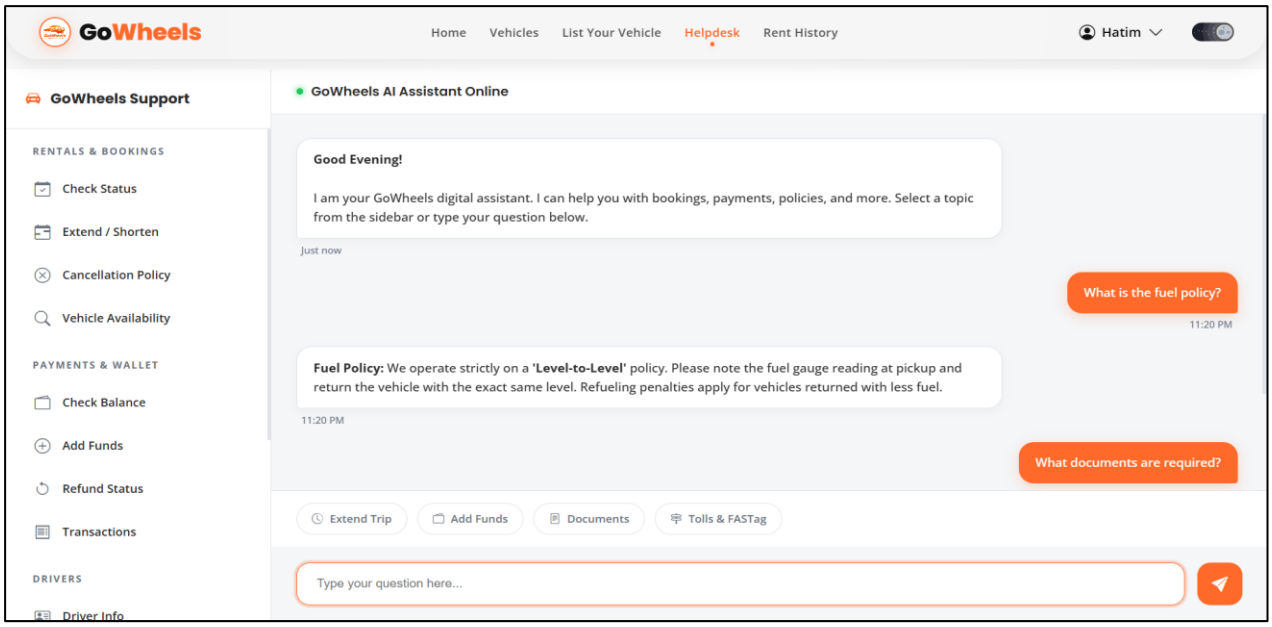

Fig 6.27: Helpdesk page

Chapter 7

Testing

7.1 Introduction

This chapter presents the testing activities carried out for the GoWheels -Online multiuser Vehicle rental and listing platform. Testing plays a critical role in ensuring that the system operates reliably and performs all intended functionalities correctly. Since the application handles vehicle bookings, user authentication, payments, and vehicle management for multiple users, any failure or incorrect behaviour in the system can lead to data inconsistency, transaction errors, or poor user experience.

The primary objective of this testing phase is to verify that all modules of the system function as expected for different user roles, including customers and vehicle owners. The system must allow valid users to register, log in securely, search and book vehicles, make payments, and manage rentals. At the same time, it should prevent unauthorized access, handle invalid inputs properly, and ensure validation of all required fields.

The testing process included manual testing as well as automated testing techniques to improve accuracy and efficiency. The following tools were used:

i. Selenium IDE: A record-and-playback tool used for testing of UI flows and basic regression testing.
ii. Selenium WebDriver: A script-based automation tool used for executing detailed test cases with validations and repeatable test execution.

Out of Scope for this Chapter:

i. Performance testing and load testing of the system
ii. Security penetration testing beyond basic validation
iii. Third-party service internal testing (e.g., payment gateway internal processing)

Overall, this chapter ensures that the GoWheels system meets its functional requirements and provides a stable and reliable platform for users.

7.2 Test Plan

System: GoWheels – Online Multiuser Vehicle Rental and Listing Platform

Version: 1.0

Test Plan ID: TP_GOWHEELS_01

i. Introduction

This test plan defines the testing strategy for the GoWheels Vehicle Rental and Listing Platform, covering both the **User Registration Module** and the **Vehicle Booking (Rent) Module**.

The purpose of this plan is to ensure that users can successfully register and book vehicles with valid inputs, while the system properly handles invalid inputs with appropriate validation messages. It also ensures data integrity, system reliability, and a smooth user experience across the platform.

ii. Test Items

i. Registration form UI elements (input fields, submit button, error messages)
ii. User input validation (Username, Email, Password)
iii. OTP verification
iv. Checkout/booking page UI elements
v. Personal details (Name, Age, Phone Number)
vi. Rental duration (Start Date, End Date)
vii. Drive option (Self-Drive / With Driver)
viii. Document upload (Aadhaar Card, Driving License)
ix. Payment methods (Wallet, UPI, Card)
x. Promo code functionality
xi. Form submission and database storage
xii. Booking confirmation process

iii. Features to be Tested

i. Registration with valid and invalid inputs
ii. Empty field validation
iii. Duplicate email/username validation
iv. Password strength validation
v. OTP verification (if implemented)
vi. Booking with valid inputs
vii. Age validation (minimum age requirement)
viii. Phone number validation

ix. Date validation (End Date > Start Date)

x. Document upload validation

xi. Drive option selection

xii. Payment processing

xiii. Promo code validation

xiv. Successful registration and booking confirmation

xv. Data storage in database

iv. Features Not to be Tested

i. Vehicle listing module (beyond booking scope)

ii. Internal payment gateway processing

iii. Email delivery delays

iv. Admin dashboard and analytics

v. Driver allocation logic

vi. Performance and load testing

v. Approach

i. Functional and non-functional testing will be performed

ii. Both positive and negative test cases will be executed

iii. Manual testing and Selenium automation will be used

iv. UI and form validation testing will be conducted

v. Backend validation and API testing (if applicable)

vi. End-to-end workflow testing (Registration → Booking → Confirmation)

vi. Pass / Fail Criteria

i. All critical and high-priority test cases must pass

ii. Minimum 95% test coverage must be achieved

iii. No critical or blocking defects should remain

iv. Expected results must match actual results

v. System should not proceed with invalid inputs

vii. Suspension Criteria and Resumption Requirements

i. Testing will be suspended if critical defects are identified

ii. Testing will resume after defects are fixed and validated

viii. Test Deliverables

i. Test plan document
ii. Test cases document
iii. Defect report
iv. Test summary report

ix. Test Tasks

i. Prepare test plan and test cases
ii. Execute test cases
iii. Identify and log defects
iv. Retest fixed defects
v. Prepare final test report

x. Test Environment

i. Operating System: Windows 10/11
ii. Browser: Google Chrome, Microsoft Edge
iii. Frontend: Django Templates (HTML, CSS, JavaScript)
iv. Backend: Django (Python)
v. Database: SQLite / MySQL
vi. Server: Localhost / Test Server
vii. Tools: Selenium IDE, Selenium WebDriver
viii. Network: Internet connection

xi. Responsibilities

i. QA Lead: Review test artifacts
ii. Test Engineer: Execute test cases and log defects
iii. Developer: Fix identified defects
iv. Project Guide/Manager: Approves final release

xii. Staffing and Training

i. Knowledge of registration and booking workflows
ii. Understanding of testing methodologies
iii. Familiarity with Selenium automation tools

xiii. Scheduling

i. Testing activities will follow the project timeline

ii. Final testing will be completed before project submission

xiv. Risks

i. Invalid or insufficient test data

ii. Payment failures or integration issues

iii. Date calculation errors

iv. Browser compatibility issues

v. Limited testing time or requirement change

xv. Approvals

i. Project Guide

ii. Project Team Members

A. Test Case 1:

Table 7: Test Case (Registration Module)

Test Case ID	Objective	Input (Steps)	Expected Result	Actual Result	Status
TC_REG_001	Registration with valid data	Enter valid username, email, password → Click Register	User should register successfully and data stored in database	User registered successfully	Pass
TC_REG_002	Registration with empty fields	Leave all fields blank → Click Register	System should display "All fields are required"	Message displayed correctly	Pass
TC_REG_003	Invalid email format	Enter email: abc123 → Click Register	System should display "Invalid email format"	Error message displayed	Pass
TC_REG_004	Weak password validation	Enter password: 12345 → Click Register	System should display "Weak password"	Message displayed correctly	Pass

TC_REG_005	Duplicate email validation	Enter already registered email → Click Register	System should display "Email already exists"	Error message displayed	Pass
TC_REG_006	Password empty	Leave password blank → Click Register	System should display "Password is required"	Message displayed correctly	Pass
TC_REG_007	Username empty	Leave username blank → Click Register	System should display "Username is required"	Message displayed correctly	Pass
TC_REG_008	Successful form submission	Fill all valid fields → Submit form	System should save data and redirect to login/dashboard	Redirected successfully	Pass
TC_REG_009	Special character validation	Enter special characters in username	System should restrict invalid characters	System handled input correctly	Pass
TC_REG_010	Multiple registration attempts	Click register multiple times rapidly	System should prevent duplicate submissions	System restricts multiple requests	Pass
TC_REG_011	Mobile/Number field validation (Failure Case)	Enter characters (abc) in numeric field → Submit	System should accept only numbers	System accepts characters also	**Fail**
TC_REG_012	OTP verification (if applicable)	Enter correct OTP → Submit	OTP should be verified successfully	OTP verified successfully	Pass

Defect Report 1:

Table 8: Defect report for numeric field

Field	Details
Defect ID	DR_REG_001
Project Name	GoWheels Vehicle Rental System
Product	Web Application
Release Version	v1.0
Module	User Registration Module
Detected Build Version	v1.1
Summary	Numeric field accepts characters instead of only numbers
Description	While executing the registration module test cases, it was observed that the numeric input field (such as mobile number/age) accepts alphabetical characters. When the user enters characters (e.g., "abc") instead of numeric values and submits the form, the system accepts the input without validation. This violates input validation rules and may lead to incorrect or invalid data being stored in the database.
Steps to Reproduce	i. Navigate to Registration Page ii. Enter valid details in all fields iii. Enter characters (e.g., "abc") in numeric field iv. Click on Register button
Actual Result	System accepts character input in numeric field
Expected Result	System should allow only numeric values and display an error message for invalid input
Severity	Medium (Data validation issue affecting data integrity)
Priority	High (Needs to be fixed to prevent invalid data entry)
Reported By	Test Engineer
Status	Open

B. Test Cases 2 :

Table 9: Test Case Booking module

Test Case ID	Objective	Input (Steps)	Expected Result	Actual Result	Status
TC_BOOK_001	Booking with valid details	Enter valid name, age (25), phone, dates → Submit	Booking should be successful	Booking completed successfully	Pass
TC_BOOK_002	Empty field validation	Leave all fields blank → Click Confirm Booking	System should display "All fields are required"	Message displayed correctly	Pass
TC_BOOK_003	Invalid phone number	Enter phone: 1234 → Submit	System should display "Invalid phone number"	Error message displayed	Pass
TC_BOOK_004	Date validation	End date before start date	System should display error	Error message displayed	Pass
TC_BOOK_005	Document upload validation	Do not upload Aadhaar/License → Submit	System should restrict booking	Validation message shown	Pass
TC_BOOK_006	Drive option selection	Select Self-Drive → Submit	System should accept selection	Selection accepted	Pass
TC_BOOK_007	Payment via wallet	Choose wallet payment → Confirm	Payment should be successful if balance available	Payment successful	Pass

TC_BOOK_008	Promo code application	Apply valid promo code	Discount should be applied	Discount applied correctly	Pass
TC_BOOK_009	UPI reference validation	Enter invalid UTR number	System should show error	Error displayed	Pass
TC_BOOK_010	Terms & Conditions validation	Do not check T&C → Submit	System should restrict booking	Message displayed	Pass
TC_BOOK_011	Age minimum validation	Enter age < 18 → Submit	System should reject underage user	Error message displayed	Pass
TC_BOOK_012	Age maximum validation (Failure Case)	Enter age = 120 → Submit	System should restrict age (max 80)	System accepts age > 80	**Fail**

Defect Report 2:

The defect report documents issues identified during the testing of the **GoWheels Vehicle Rental and Listing Platform**. It provides detailed information about defects, including their description, severity, priority, and reproduction steps. This helps the development team to identify, analyze, and resolve issues efficiently while ensuring system quality and reliability.

Table 10: Defect report for age input

Field	Details
Defect ID	DR_BOOK_001
Project Name	GoWheels Vehicle Rental System
Product	Web Application
Release Version	v1.0
Module	Booking Module

Detected Build Version	v1.1
Summary	System accepts age greater than allowed maximum limit
Description	During execution of booking module test cases, it was observed that the system accepts age values greater than the defined maximum limit (80 years). When a user enters age as 120 and submits the booking form, the system processes the booking successfully instead of restricting it. This violates the business rule for age validation and may allow invalid users to proceed with booking.
Steps to Reproduce	i. Navigate to Booking Page ii. Enter all valid booking details iii. Enter age as 120 iv. Click on Confirm Booking
Actual Result	System accepts age greater than 80 and completes booking
Expected Result	System should restrict age above 80 and display an appropriate error message
Severity	Medium (Validation issue affecting business rules)
Priority	High (Needs to be fixed to enforce proper validation)
Reported By	Test Engineer
Status	Open

Chapter 8
Conclusion and Future scope

8.1 Conclusion

In conclusion, the "GoWheels" application provides a highly efficient, centralized, and secure platform to manage the entire lifecycle of vehicle rentals. By digitizing the traditionally fragmented and manual rental market, the system bridges the gap between vehicle owners and individuals seeking short-term transportation.

By integrating features such as digital KYC verification, real-time availability tracking, an internal wallet system, and the flexibility to choose between self-drive or chauffeur-driven options, the platform significantly reduces administrative overhead and enhances user trust. Overall, GoWheels serves as a highly beneficial, modernized digital marketplace that optimizes inventory management for host and ensures a seamless, transparent booking experience for renters.

8.2 Future Scope

While the current system successfully fulfills its primary objectives, there are several avenues for future enhancement to scale the platform and introduce advanced technological integrations:

i. **Mobile Application Development:** The web-based platform can be expanded into dedicated mobile applications for Android and iOS (using frameworks like Flutter or React Native) to provide users with a more native and accessible experience.

ii. **AI-Driven Dynamic Pricing:** Introducing machine learning algorithms to automatically adjust rental prices based on real-time market demand, festive seasons, and vehicle availability.

iii. **Geographical Expansion:** The platform can be scaled to operate pan-India, supporting multiple cities and incorporating multi-lingual support to cater to diverse regional users.

iv. **Automated Insurance Processing:** Integrating a module to offer instant, short-term trip insurance during the checkout process and facilitating automated claim workflows in the event of an accident.

Chapter 9
References and Bibliography

References:-

Vehicle Rental Agency Owners and Independent Drivers with whom we discussed the requirements as well as the knowledge of the local rental workflow:

i. Shri. Ramesh Patil (Owner, Local Tours & Travels)
ii. Shri. Santosh Sonawane (Independent Vehicle Owner)
iii. Shri. Vikas Wagh (Professional Chauffeur/Driver)
iv. Shri. Deepak Chaudhari (Independent Vehicle Owner)
v. Shri. Prakash Deore (Professional Chauffeur/Driver)

Existing Vehicle Rental Applications Studied:

i. **Zoomcar** Developed by Zoomcar India Pvt. Ltd., representing the standard for self-drive car rentals in India.

ii. **Revv** Developed by PrimeMover Mobility Technologies Pvt. Ltd., studied for their flexible booking and delivery models.

iii. **Turo** Studied for its successful implementation of the peer-to-peer vehicle sharing and hosting model.

Technical Handbooks and Documentation:

i. **Django 4** By Antonio Melé

ii. **Python Crash Course** (A Hands-On, Project-Based Introduction to Programming) Eric Matthes

iii. **Official Django Documentation** (https://docs.djangoproject.com/)

iv. **Stripe API Official Documentation** (https://stripe.com/docs) - Studied for integrating the secure digital wallet and payment gateway.

v. **Bootstrap 5 Official Documentation** (https://getbootstrap.com/) - Studied for designing the responsive, mobile-friendly frontend user interface.